Sparse Multivariate Rational Runction Interpolation

by

Archit Srivastava

M.S., University of Nebraska, Lincoln 2022
B.Tech., Dr. Abul Kalam Azad Technical University, 2017

Thesis Submitted in Partial Fulfillment of the
Requirements for the Degree of
Master of Science

in the
Department of Mathematics
Faculty of Example Names

SIMON FRASER UNIVERSITY
Fall 2025

Declaration of Committee

Name:	**Archit Srivastava**
Degree:	**Master of Science**
Thesis title:	**Sparse Multivariate Rational Runction Interpolation**
Committee:	**Chair:** Michael Monagan Professor, Mathematics

Abstract

Abstract paragraphs should be unindented. Master's abstracts are limited to 150 words; the limit is 350 words for doctoral abstracts. Abstract text must fit on a single page.

Keywords: thesis template; Simon Fraser University; LaTeX time travel paradoxes

Dedication

This is an optional page. Use your choice of paragraph style for text on this page.

Acknowledgements

This is an optional page. Use your choice of paragraph style for text on this page.

Table of Contents

Declaration of Committee ii

Abstract iii

Dedication iv

Acknowledgements v

Table of Contents vi

List of Tables ix

List of Figures x

1 Introduction 1

1.1 A brief overview of computer arithmetic 1

1.2 Cost of integer arithmetic operations 2

1.2.1 Addition 2

1.2.2 Multiplication 2

1.3 Performance issue: Intermediate expression swell 2

1.3.1 Rational Reconstruction Bound 6

1.4 Black boxes 9

1.5 Polynomial black boxes and evaluation homomorphisms 9

1.5.1 Black boxes for multivariate polynomial 9

1.5.2 Modular black boxes 10

1.5.3 Modular black boxes for rational functions 10

1.6 Randomized algorithm, early termination 10

1.6.1 Randomized algorithms 10

1.6.2 Monte Carlo algorithms 10

1.6.3 Failure probability 11

1.6.4 Early termination 11

1.7 Sparse polynomials 12

1.7.1 Sparse polynomial definition and example 12

1.7.2 Sparse polynomial interpolation Ben-Or Tiwari 12
1.7.3 Sparse rational function interpolation Kaltofen Yang 12
1.7.4 Application 12

2 Maximal Quotient Rational Function Reconstruction 15
2.1 Univariate rational function reconstruction 15
2.2 The extended Euclidean algorithm 15
2.3 The Chinese remainder theorem 16
2.4 Rational function reconstruction 16
2.5 Maximal quotient rational function reconstruction 18
2.6 Failure probability 20

3 Ben-Or Tiwari Interpolation 21
3.1 Introduction 21
3.2 Linear recursive sequences 21
3.2.1 Linear recurrence as a linear system 21
3.2.2 Characteristic polynomial of the companion matrix 23
3.2.3 Eigen values of the companion matrix are the general solution of the linear recurrence 25
3.2.4 Hankel matrix 27
3.2.5 Polynomials as linear recurrences. 28
3.3 Berlekamp Massey algorithm 31
3.3.1 Equivalence of Berlekamp Massey algorithm and extended Euclidean algorithm 33
3.4 Polynomial evaluation as dot product 34
3.5 Polynomial interpolation as a linear transform 34
3.5.1 Zippel transposed Vandermonde solver 35
3.6 Ben-Or and Tiwari's multivariate polynomial interpolation 37
3.7 Pseudocode 37
3.8 Example of Ben-Or Tiwari multivariate interpolation 40
3.9 Failure probability 42

4 Sparse Multivariate Rational function interpolation 43
4.1 Kaltofen Yang algorithm 43
4.2 Pseudocode 45
4.3 Example of Kaltofen Yang multivariate rational function interpolation . . . 48

5 Solving Parametric Linear Systems using Sparse Multivariate Rational Function Interpolation 56

Bibliography 57

Appendix A Code **59**

List of Tables

Table 2.1 Extended Euclidean algorithm computations for input polynomials $\overline{m}$ and u (over $\mathbb{Z}_{19}$). 20

Table 4.1 Computed values of σ_1^0, σ_2^0, and $T_0(\alpha_j)$ 49
Table 4.2 MQRFR computations for input polynomials $\overline{m}(x)$ and $u_0(x)$ 50
Table 4.3 Computed values of σ_1^1, σ_2^1, and $T_1(\alpha_j)$ 50
Table 4.4 Values of σ_i, Φ_i, u_i, $N_i(x), D_i(x)$ and related quantities. 51
Table 4.5 Values of σ^i, $N_i(\sigma^i)$, and $D_i(\sigma^i)$. 52

List of Figures

Figure 4.1 High level overview of Kaltofen Yang algorithm 43

Chapter 1

Introduction

We know that given n points in $\mathbb{R}^2$ we can fit a polynomial of degree $n-1$ through it, however, sometimes it is better to fit a rational function over the given points. Rational functions have more flexibility and can model more complex behavior such as, asymptotes, poles, saturation effects, decay rates. This often gives better global accuracy than polynomials [15].
In this work, we will explore the Kaltofen-Yang sparse multivariate rational function interpolation algorithm [10]. Given $d+1$ points in an n dimensional space $\mathbb{Q}^n$, the objective is to find a multivariate rational function in $\mathbb{Q}(x_1, \ldots, x_n)$.

1.1 A brief overview of computer arithmetic

There are multiple computational paradigms for performing arithmetic on modern computers. The following are the most widely used

- Fixed point arithmetic.
- Floating point arithmetic, defined by the IEEE 754 standard.
- Multi-precision arithmetic.

Numerical computation uses floating point arithmetic for approximations which are supported by general purpose hardware like CPUs and GPUs.

Computer algebra systems use multi-precision arithmetic paradigm for computing exact values. Multi-precision arithmetic is not supported by general purpose hardware and is implemented through software that is often optimized at assembly level for different processors. Multi-precision arithmetic has no universal standard like IEEE 754, but GNU's GMP (GNU Multiple Precision Arithmetic Library) and MPFR (GNU Multiple Precision Floating-Point Reliable Library) are the most widely used multi-precision libraries and act

as functional standards. Multi-precision libraries are the bedrock for computer algebra systems (Maple), cryptography libraries (OpenSSL, libgcrypt), and computational geometry library (CGAL).

1.2 Cost of integer arithmetic operations

We now briefly mention the bit complexity bounds in terms of the number of bits required to represent the operands using the big O notation for some algorithms for performing integer addition and multiplication. They provide information on how the computation cost scales as the size of operands increases. The bit-complexity of two n bit integers for different algorithms for addition and multiplication are as follows.

1.2.1 Addition

Adding two n bit integers has bit complexity of $O(n)$ [12].

1.2.2 Multiplication

The cost of multiplying two integers depends on the chosen multiplication algorithm. Several algorithms with different asymptotic complexities are listed below.

1. The Naïve multiplication algorithm has a bit complexity of $O(n^2)$.
2. Karatsuba multiplication is a divide-and-conquer algorithm that reduces the multiplication of two n-digit integers to three multiplications of half-size numbers with bit complexity $O(n^{\log_2 3}) \approx O(n^{1.585})$ [11].
3. Toom-Cook [13] multiplication generalizes Karatsuba's algorithm. It divides a large integer into smaller parts, uses divide and conquer, and polynomial interpolation to achieve the asymptotic bit complexity of $O(n^{1.465})$.
4. Schönhage–Strassen [17] FFT multiplication uses fast Fourier transforms over modular rings to achieve $O(n \log n \log \log n)$.
5. Fürer [4] improved the asymptotic bound to $O(n \log n \, 2^{O(\log^* n)})$.
6. Harvey and van der Hoeven [7] improved the bound to $O(n \log n)$. which is currently the best known asymptotic bound.

1.3 Performance issue: Intermediate expression swell

Multi-precision libraries used in computer algebra systems provide fast, optimized implementations to perform arithmetic over $\mathbb{Z}$, $\mathbb{Q}$ and $\mathbb{Z}_p$. Moreover, unlike fixed point arithmetic,

which results in overflow when the size of the number exceeds 32 bits in case of single precision and 64 bits in case of double precision, multi-precision libraries do not have an upper bound for the size of the operands and the result. However, this results in a blow up in the size of operands and intermediate results, slowing down computation. Since polynomial interpolation is a linear operator, and we will apply Kaltofen-Yang sparse multivariate rational function interpolation to solve a parametric system of linear equations, it is fitting to consider the following example from [5] to demonstrate expression swell.

Example 1. *Solve a system of linear equations with integer coefficients, using Gaussian elimination using only integer arithmetic,*

$$\begin{aligned} 22x + 44y + 74z &= 1, \\ 15x + 14y - 10z &= -2, \\ -25x - 28y + 20z &= 34. \end{aligned}$$

Gaussian elimination with integers.

$$\left[\begin{array}{ccc|c} 22 & 44 & 74 & 1 \\ 15 & 14 & -10 & -2 \\ -25 & -28 & 20 & 34 \end{array}\right] \xrightarrow{R_2 \leftarrow 22R_2 - 15R_1} \left[\begin{array}{ccc|c} 22 & 44 & 74 & 1 \\ 0 & -352 & -1330 & -59 \\ -25 & -28 & 20 & 34 \end{array}\right]$$

$$\left[\begin{array}{ccc|c} 22 & 44 & 74 & 1 \\ 0 & -352 & -1330 & -59 \\ -25 & -28 & 20 & 34 \end{array}\right] \xrightarrow{R_3 \leftarrow 25R_1 + 22R_3} \left[\begin{array}{ccc|c} 22 & 44 & 74 & 1 \\ 0 & -352 & -1330 & -59 \\ 0 & 484 & 2290 & 773 \end{array}\right]$$

$$\left[\begin{array}{ccc|c} 22 & 44 & 74 & 1 \\ 0 & -352 & -1330 & -59 \\ 0 & 484 & 2290 & 773 \end{array}\right] \xrightarrow{\substack{R_2 \leftarrow 11R_2, \\ R_3 \leftarrow 8R_3, \\ R_3 \leftarrow R_3 + R_2}} \left[\begin{array}{ccc|c} 22 & 44 & 74 & 1 \\ 0 & -3872 & -14630 & -649 \\ 0 & 0 & 3690 & 5535 \end{array}\right]$$

$$\left[\begin{array}{ccc|c} 22 & 44 & 74 & 1 \\ 0 & -3872 & -14630 & -649 \\ 0 & 0 & 3690 & 5535 \end{array}\right] \xrightarrow{R_2 \leftarrow 3690R_2 + 1330R_3} \left[\begin{array}{ccc|c} 22 & 44 & 74 & 1 \\ 0 & -1298880 & 0 & 7143840 \\ 0 & 0 & 3690 & 5535 \end{array}\right]$$

$$\left[\begin{array}{ccc|c} 22 & 44 & 74 & 1 \\ 0 & -1298880 & 0 & 7143840 \\ 0 & 0 & 3690 & 5535 \end{array}\right] \xrightarrow{R_1 \leftarrow 1298880R_1 + 44R_2} \left[\begin{array}{ccc|c} 28575360 & 0 & 96017120 & 315627840 \\ 0 & -1298880 & 0 & 7143840 \\ 0 & 0 & 3690 & 5535 \end{array}\right]$$

$$\left[\begin{array}{ccc|c} 28575360 & 0 & 96017120 & 315627840 \\ 0 & -1298880 & 0 & 7143840 \\ 0 & 0 & 3690 & 5535 \end{array}\right] \xrightarrow{R_1 \leftarrow 3690R_1 - 74R_3} \left[\begin{array}{ccc|c} 1257315840 & 0 & 0 & 7543895040 \\ 0 & -1298880 & 0 & 7143840 \\ 0 & 0 & 3690 & 5535 \end{array}\right]$$

$$\implies 1257315840x = 7543895040,$$
$$\implies -57150720y = 314328960,$$
$$\implies 162360z = 243540.$$

$$\implies x = \frac{7543895040}{1257315840} = 6,$$
$$\implies y = \frac{314328960}{-57150720} = -\frac{11}{2},$$
$$\implies z = \frac{243540}{162360} = \frac{3}{2}.$$

Even though the final result is small, we end up with some pretty large intermediate values that slow down the computation.

Since the final solution is over $\mathbb{Q}^3$, we can allow fractions while performing row operations to avoid blowing up of the size of intermediate entries.

Gaussian elimination with fractions.

$$\left[\begin{array}{ccc|c} 22 & 44 & 74 & 1 \\ 15 & 14 & -10 & -2 \\ -25 & -28 & 20 & 34 \end{array}\right] \xrightarrow{R_1 \leftarrow \frac{1}{22} R_1} \left[\begin{array}{ccc|c} 1 & 2 & \frac{37}{11} & \frac{1}{22} \\ 15 & 14 & -10 & -2 \\ -25 & -28 & 20 & 34 \end{array}\right]$$

$\gcd(37, 11) = \gcd(1, 22) = 1$

$$\left[\begin{array}{ccc|c} 1 & 2 & \frac{37}{11} & \frac{1}{22} \\ 15 & 14 & -10 & -2 \\ -25 & -28 & 20 & 34 \end{array}\right] \xrightarrow[R_3 \leftarrow R_3 + 25R_1]{R_2 \leftarrow R_2 - 15R_1} \left[\begin{array}{ccc|c} 1 & 2 & \frac{37}{11} & \frac{1}{22} \\ 0 & -16 & -\frac{665}{11} & -\frac{59}{22} \\ 0 & 22 & \frac{1145}{11} & \frac{773}{22} \end{array}\right]$$

$\gcd(665, 11) = \gcd(59, 22), \gcd(1145, 11) = \gcd(773, 22) = 1$

$$\left[\begin{array}{ccc|c} 1 & 2 & \frac{37}{11} & \frac{1}{22} \\ 0 & -16 & -\frac{665}{11} & -\frac{59}{22} \\ 0 & 22 & \frac{1145}{11} & \frac{773}{22} \end{array}\right] \xrightarrow{R_2 \leftarrow -\frac{1}{16} R_2} \left[\begin{array}{ccc|c} 1 & 2 & \frac{37}{11} & \frac{1}{22} \\ 0 & 1 & \frac{665}{176} & \frac{59}{352} \\ 0 & 22 & \frac{1145}{11} & \frac{773}{22} \end{array}\right]$$

$\gcd(665, 176) = \gcd(59, 352) = 1.$

$$\left[\begin{array}{ccc|c} 1 & 2 & \frac{37}{11} & \frac{1}{22} \\ 0 & 1 & \frac{665}{176} & \frac{59}{352} \\ 0 & 22 & \frac{1145}{11} & \frac{773}{22} \end{array}\right] \xrightarrow[R_3 \leftarrow R_3 - 22R_2]{R_1 \leftarrow R_1 - 2R_2} \left[\begin{array}{ccc|c} 1 & 0 & -\frac{369}{88} & -\frac{51}{176} \\ 0 & 1 & \frac{665}{176} & \frac{59}{352} \\ 0 & 0 & \frac{1845}{88} & \frac{5535}{176} \end{array}\right]$$

$\gcd(369, 88) = \gcd(51, 176) = \gcd(1845, 88) = \gcd(5535, 176) = 1.$

$$\left[\begin{array}{ccc|c} 1 & 0 & -\frac{369}{88} & -\frac{51}{176} \\ 0 & 1 & \frac{665}{176} & \frac{59}{352} \\ 0 & 0 & \frac{1845}{88} & \frac{5535}{176} \end{array}\right] \xrightarrow{R_3 \leftarrow \frac{88}{1845} R_3} \left[\begin{array}{ccc|c} 1 & 0 & -\frac{369}{88} & -\frac{51}{176} \\ 0 & 1 & \frac{665}{176} & \frac{59}{352} \\ 0 & 0 & 1 & \frac{487080}{324720} \end{array}\right]$$

$\gcd(487080, 324720) = 162360$.

Normalization of the fraction:

$$\frac{487080 \frac{1}{162360}}{324720 \frac{1}{162360}} = \frac{3}{2}$$

$$\left[\begin{array}{ccc|c} 1 & 0 & -\frac{369}{88} & -\frac{51}{176} \\ 0 & 1 & \frac{665}{176} & \frac{59}{352} \\ 0 & 0 & 1 & \frac{3}{2} \end{array}\right] \xrightarrow{R_2 \leftarrow R_2 - \frac{665}{176} R_3} \left[\begin{array}{ccc|c} 1 & 0 & -\frac{369}{88} & -\frac{51}{176} \\ 0 & 1 & 0 & -\frac{1936}{352} \\ 0 & 0 & 1 & \frac{3}{2} \end{array}\right]$$

$\gcd(1936, 352) = 176$.

$$\left[\begin{array}{ccc|c} 1 & 0 & -\frac{369}{88} & -\frac{51}{176} \\ 0 & 1 & 0 & -\frac{11}{2} \\ 0 & 0 & 1 & \frac{3}{2} \end{array}\right] \xrightarrow{R_1 \leftarrow R_1 + \frac{369}{88} R_3} \left[\begin{array}{ccc|c} 1 & 0 & 0 & \frac{1056}{176} \\ 0 & 1 & 0 & -\frac{11}{2} \\ 0 & 0 & 1 & \frac{3}{2} \end{array}\right]$$

$\gcd(1056, 176) = 176$

Normalization of the fraction:

$$\frac{1056 \frac{1}{176}}{176 \frac{1}{176}} = 6$$

Thus, we recovered the solution of the system

$$x = 6, \qquad y = -\frac{11}{2}, \qquad z = \frac{3}{2}.$$

We see that performing arithmetic over rational numbers can also lead to a larger expression as we get large fractions like $\frac{487080}{324720}$ in the matrix. Moreover, addition and multiplication operations on the rationals are slow. For adding two fractions, we first find the common denominator that takes one multiplication, scale the numerators, that is, two more multiplications, then there is an addition, a gcd computation and if the gcd $\neq 1$, two divisions to get the fraction in the simplest form. Similarly, for fraction multiplication, there are two multiplications (for the numerator and denominator), a gcd computation, and if the gcd $\neq 1$, there are two divisions to get the fraction in the simplest form. So, using rationals is also inefficient. We therefore use modular arithmetic in a finite field $\mathbb{Z}_p$. It solves has two advantages.

1. The elements of the field $\mathbb{Z}_p$ are less than p so that bounds the size of the operands from above.

2. Performing additions and multiplications on $\mathbb{Z}_p$ does not require as many computations as performing additions and multiplications on $\mathbb{Q}$.

After doing all the arithmetic, we can use the Chinese remainder theorem with the extended Euclidean algorithm to recover the rational pre-images from the modular images of the result.

1.3.1 Rational Reconstruction Bound

Suppose we want to recover a reduced rational number

$$\frac{n}{d} \in \mathbb{Q}, \qquad \gcd(n, d) = 1, \quad d > 0,$$

from its modular residue

$$a \equiv nd^{-1} \pmod{p},$$

where p is a prime modulus. The classical rational reconstruction theorem states that the pair (n, d) is *uniquely* reconstructible whenever

$$|n| \leq B, \qquad |d| \leq B,$$

and the modulus p satisfies

$$B^2 < \frac{p}{2}.$$

Equivalently,

$$p > 2B^2.$$

This bound guarantees that among all rational numbers whose numerator and denominator do not exceed B in absolute value, at most one of them can map to the same residue class modulo p. Therefore the Euclidean–algorithm-based rational reconstruction algorithm (see [5]) can recover the original fraction uniquely from a single modular evaluation.

Application to $-11/2$

For the rational number

$$\frac{n}{d} = -\frac{11}{2},$$

we have

$$|n| = 11, \qquad |d| = 2.$$

Thus the maximum bound is

$$B = \max(|n|, |d|) = 11.$$

To guarantee unique rational reconstruction, we require

$$p > 2B^2 = 2 \cdot 11^2 = 242.$$

Hence any prime $p > 242$ suffices. The smallest such prime is

$$p = 251$$

Modular Gaussian elimination over $\mathbb{Z}_p$, where $p = 251$

Th augmented matrix over $\mathbb{Z}_{251}$ is

$$\left[\begin{array}{ccc|c} 22 & 44 & 74 & 1 \\ 15 & 14 & 241 & 249 \\ 226 & 223 & 20 & 34 \end{array}\right],$$

since $-10 \equiv 241$, $-2 \equiv 249$, $-25 \equiv 226$, and $-28 \equiv 223 \pmod{251}$.

$$22^{-1} \equiv 194 \pmod{251}$$

$$\left[\begin{array}{ccc|c} 22 & 44 & 74 & 1 \\ 15 & 14 & 241 & 249 \\ 226 & 223 & 20 & 34 \end{array}\right] \xrightarrow{R_1 \leftarrow 194\, R_1} \left[\begin{array}{ccc|c} 1 & 2 & 49 & 194 \\ 15 & 14 & 241 & 249 \\ 226 & 223 & 20 & 34 \end{array}\right].$$

$$\left[\begin{array}{ccc|c} 1 & 2 & 49 & 194 \\ 15 & 14 & 241 & 249 \\ 226 & 223 & 20 & 34 \end{array}\right] \xrightarrow[R_3 \leftarrow R_3 - 226R_1]{R_2 \leftarrow R_2 - 15R_1} \left[\begin{array}{ccc|c} 1 & 2 & 49 & 194 \\ 0 & 235 & 8 & 100 \\ 0 & 22 & 241 & 115 \end{array}\right].$$

$$235^{-1} \equiv 47 \pmod{251}$$

$$\left[\begin{array}{ccc|c} 1 & 2 & 49 & 194 \\ 0 & 235 & 8 & 100 \\ 0 & 22 & 241 & 115 \end{array}\right] \xrightarrow{R_2 \leftarrow 47\, R_2} \left[\begin{array}{ccc|c} 1 & 2 & 49 & 194 \\ 0 & 1 & 125 & 182 \\ 0 & 22 & 241 & 115 \end{array}\right].$$

$$\left[\begin{array}{ccc|c} 1 & 2 & 49 & 194 \\ 0 & 1 & 125 & 182 \\ 0 & 22 & 241 & 115 \end{array}\right] \xrightarrow[R_3 \leftarrow R_3 - 22R_2]{R_1 \leftarrow R_1 - 2R_2} \left[\begin{array}{ccc|c} 1 & 0 & 50 & 81 \\ 0 & 1 & 125 & 182 \\ 0 & 0 & 1 & 127 \end{array}\right].$$

$$\left[\begin{array}{ccc|c} 1 & 0 & 50 & 81 \\ 0 & 1 & 125 & 182 \\ 0 & 0 & 1 & 127 \end{array}\right] \xrightarrow[R_2 \leftarrow R_2 - 125R_3]{R_1 \leftarrow R_1 - 50R_3} \left[\begin{array}{ccc|c} 1 & 0 & 0 & 6 \\ 0 & 1 & 0 & 120 \\ 0 & 0 & 1 & 127 \end{array}\right].$$

Thus, over $\mathbb{Z}_{251}$, the solution is

$$x \equiv 6, \qquad y \equiv 120, \qquad z \equiv 127 \pmod{251}.$$

1.4 Black boxes

In computer algebra, the "black box model" refers primarily to an implicit computational model for an arbitrary element over some algebraic object [9]. Consider the following examples over some commutative ring R and field $\mathbb{F}$ whose black box is implemented as a procedure in a computer algebra system that accepts appropriate inputs from the user to pick a particular element from the underlying algebraic object, perform the appropriate computations and return the output.

- Value of $f(\alpha_1, \ldots, \alpha_n) \in R$ or $\mathbb{F}$ for some polynomial $f(x_1, \ldots, x_n) \in R[x_1, \ldots, x_n]$ or $\mathbb{F}[x_1, \ldots, x_n]$ at some point $\alpha \in R^n$ or $\mathbb{F}^n$.
- Value of $\frac{f(\alpha_1,\ldots,\alpha_n)}{g(\alpha_1,\ldots,\alpha_n)} \in \mathbb{F}$ for some rational function $\frac{f(x_1,\ldots,x_n)}{g(x_1,\ldots,x_n)}$ over a field of rational functions $\mathbb{F}(x_1, \ldots, x_n)$.
- Determinant $Det(A) \in R[y_1, \ldots, y_m]$ or $\mathbb{F}[y_1, \ldots, y_m]$ of some matrix $A \in M_n(R[y_1, \ldots, y_m])$, or $M_n(\mathbb{F}[y_1, \ldots, y_m])$.
- Solution vector x for some parametric system of linear equations $Ax = b$, where $A \in M_n(R[y_1, \ldots, y_m])$, or $M_n(\mathbb{F}[y_1, \ldots, y_m])$ and $b \in R^n$ or $\mathbb{F}^n$.

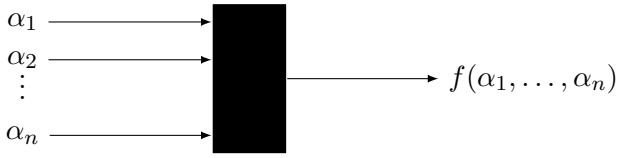
The black box $\mathcal{B}$ of an algebraic object hides its internal structure and complexity.

emulates the underlying mathematical properties of the are equivalent to the evaluation homomorphisms. Probing or querying the black box of an object produces its image under the evaluation homomorphism. Thus, the black box model works like a transform (like the Fourier transform, Cosine transform, etc.) that generates images that may be more efficient or easier to work with.

1.5 Polynomial black boxes and evaluation homomorphisms

1.5.1 Black boxes for multivariate polynomial

Definition 1.5.1. *Let f be a polynomial in n variables $x_1, \ldots, x_n$ over a field $\mathbb{F}$ ie. $f \in \mathbb{F}[x_1, \ldots, x_n]$. A black box $\mathcal{B}$ for f is a program that on input $\alpha \in \mathbb{F}^n$ computes $f(\alpha)$ ie. $\mathcal{B} : \mathbb{F}^n \longrightarrow \mathbb{F}$.*



The following theorem from Dummit and Foote

Theorem 1.5.1. *[3] Let $\mathbb{F}[x]$ be a ring of polynomials over a field $\mathbb{F}$ and let $I \subset \mathbb{F}[x]$ be an ideal of the ring, then $\mathbb{F}[x]/I$ is a field if and only if I is a maximal ideal.*

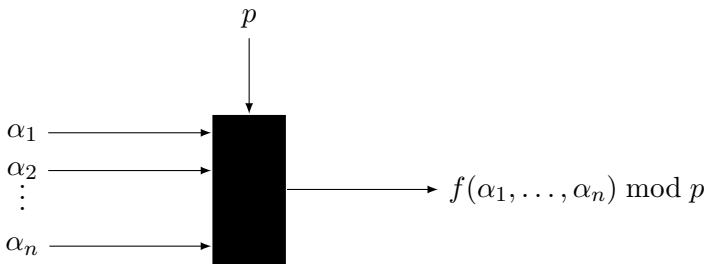
This theorem also holds for the ring of multivariate polynomials $\mathbb{F}[x_1, \ldots, x_n]$.

Definition 1.5.2. *Let $\mathbb{F}$ be a field and $\mathbb{F}[x_1, \ldots, x_n]$ be the ring of polynomials with n variables. Let $f(x_1, \ldots, x_n) \in \mathbb{F}[x_1, \ldots, x_n]$ and $\alpha \in \mathbb{F}^n$ be a point in the field. The evaluation homomorphism is a map*

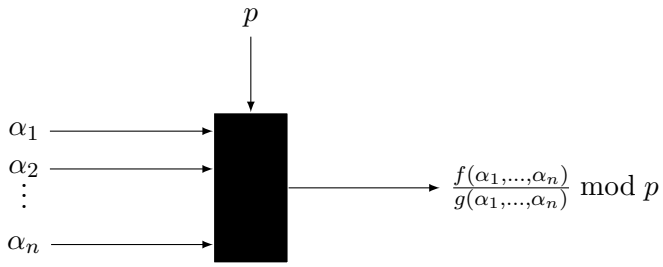
$$\begin{array}{rccc} \phi: & \mathbb{F}[x_1, \ldots, x_n] & \longrightarrow & \mathbb{F}[x_1, \ldots, x_n]/(x_1 - \alpha_1, \ldots, x_n - \alpha_n) \cong \mathbb{F} \\ & f(x_1, \ldots, x_n) & \longmapsto & f(\alpha_1, \ldots, \alpha_n) \end{array}$$

Let $\mathcal{B}$ be the black box for $f(x) \in \mathbb{F}[x]$, probing or querying $\mathcal{B}$ at α is the same as taking the image under the evaluation homomorphism ϕ.

1.5.2 Modular black boxes



1.5.3 Modular black boxes for rational functions



1.6 Randomized algorithm, early termination

1.6.1 Randomized algorithms

The black box model introduces randomness since the points we query the black box with can be randomly generated. This introduces a new challenge, what if the point we query the black box with is in the kernel of the homomorphism. This will generate a zero, which may be undesirable depending on the context. Thus, depending on the context an algorithm using black boxes may fail with non-zero probability.

1.6.2 Monte Carlo algorithms

Monte Carlo algorithms use repeated random sampling to produce output in finite time, however they have non zero failure probability.

For an algorithm using black boxes, failure occurs when one of the sampled points is in the kernel of the underlying homomorphism of the black box. Whenever the algorithm fails, we can choose a different point, get the image of the object at that point, and rerun the algorithm.

1.6.3 Failure probability

If we have a black box for a non-zero polynomial over a finite field $\mathbb{F}_p$, then the kernel of the evaluation homomorphism will be the roots of the polynomial, and its failure probability is given by the Schwartz-Zipple lemma.

Lemma 1.6.1 (Schwartz Zippel lemma)**.** *Let $f(x_1, \ldots, x_n) \in \mathbb{F}_p[x_1, \ldots, x_n]$ be such that $f \neq 0$ and $S \subset \mathbb{F}_p$ be large and finite. If we randomly sample points α from S then*

$$Pr[failure] = Pr[f(s_i) = 0] = \frac{degree(f)}{|S|}$$

If we choose a large prime p, and let S be equal to $\mathbb{F}_p$, we can make the failure probability as low as we desire, thus making sure that the algorithm gives the correct output with high probability (whp). In case we do end up with an unlucky point we acn

1.6.4 Early termination

While using black boxes for polynomial interpolation, we loose information about the polynomials like its degree and the number of terms it contains. This makes it challenging to come up with a bound that acts as a termination condition beforehand for an iterative algorithm using the black box.
Numerical algorithms use the notion of convergence of error as a termination condition. We need something analogous for algorithms that use black boxes for polynomials.
Early termination is a probabilistic technique in black-box sparse polynomial interpolation that allows algorithms to adaptively detect and terminate once the polynomial is interpolated, without requiring an a priori upper bound $\tau \geq t$. [8] This dramatically reduces black-box probes to the black box.

Let $f(x) \in \mathbb{F}_p[x]$ and $S \subset \mathbb{F}_p$ be large and finite. We randomly sample points s_i from S for polynomial interpolation. Once the correct interpolating polynomial $\hat{f}$ has been reconstructed, future probes satisfy $f(\alpha) = \hat{f}(\alpha)$ whp, probability. Thus, we can terminate the algorithm once the output stabilizes.

1.7 Sparse polynomials

1.7.1 Sparse polynomial definition and example

Definition 1.1. Let f be a non-zero polynomial in $y_1, y_2, \ldots, y_m$. Let t denote the number of non-zero terms in f such that $t \geq 1$ and let $d = \deg(f)$ be the total degree of f. The maximum number of possible terms in f is

$$M = \binom{m+d}{d}.$$

We say that f is *sparse* if $t < \sqrt{M}$.

Example 1.2. Let $f = 3y_1y_2y_5 + 5y_1y_3^3y_4$. Notice that $t = 2$, $d = 5$, $m = 5$, and

$$M = \binom{5+5}{5} = \binom{10}{5} = 252.$$

Therefore, f is a sparse polynomial since

$$t = 2 < \sqrt{M} = \sqrt{252} \approx 15.9.$$

Definition 1.3. A sparse polynomial f is normally represented as

$$f = \sum_{k=1}^{t} a_k \, y_1^{d_{k,1}} y_2^{d_{k,2}} \cdots y_m^{d_{k,m}}, \qquad \text{where } a_k \neq 0.$$

Definition 1.4. A multivariate rational function f/g with $\gcd(f, g) = 1$ is *sparse* if both polynomials f and g are sparse.

1.7.2 Sparse polynomial interpolation Ben-Or Tiwari

1.7.3 Sparse rational function interpolation Kaltofen Yang

1.7.4 Application

1.2.2 Solving $Ax = b$ using sparse rational function interpolation

Consider the parametric linear system $Ax = b$ where

$$A \in \mathbb{Z}[y_1, y_2, \ldots, y_m]^{n \times n}$$

is the coefficient matrix of full rank n and

$$b \in \mathbb{Z}[y_1, y_2, \ldots, y_m]^n$$

is the right-hand side column vector such that the number of terms in the entries of A and b, denoted by $\#A_{ij}, \#b_i$, is $\leq t$ and $\deg(A_{ij}), \deg(b_i) \leq d$.

Elementary linear algebra tells us that the solution vector x is unique since the coefficient matrix A is of full rank n. In this thesis, we aim to interpolate the solution vector of rational functions

$$x = \begin{bmatrix} x_1 & x_2 & \cdots & x_n \end{bmatrix}^T = \begin{bmatrix} \frac{f_1}{g_1} & \frac{f_2}{g_2} & \cdots & \frac{f_n}{g_n} \end{bmatrix}^T \tag{1.1}$$

such that for $f_k, g_k \in \mathbb{Z}[y_1, y_2, \ldots, y_m]$, $g_k \neq 0$, $g_k \mid \det(A)$, and $\gcd(f_k, g_k) = 1$ for $1 \leq k \leq n$.

Using Cramer's rule, the solutions of $Ax = b$ are given by

$$x_i = \frac{\det(A^i)}{\det(A)} \in \mathbb{Z}(y_1, \ldots, y_m) \tag{1.2}$$

where A^i is the matrix obtained by replacing the i-th column of A with b, and $\det(A)$ is a polynomial in $\mathbb{Z}[y_1, y_2, \ldots, y_m]$.

Let

$$\tilde{x}_i := \det(A^i) = x_i \det(A) \in \mathbb{Z}[y_1, y_2, \ldots, y_m].$$

Maple and other computer algebra systems such as Magma have an implementation of the Bareiss/Edmonds one-step fraction-free Gaussian elimination algorithm [**?**, **?**], which triangularizes an augmented matrix $B = [A \mid b]$ to obtain $\det(A)$ as a polynomial in $\mathbb{Z}[y_1, y_2, \ldots, y_m]$, and then solves for the polynomials $\tilde{x}_i$ via back substitution using Lipson's fraction-free back formula [**?**]. Ignoring pivoting, the following pseudocode (Algorithm 1) of the Bareiss/Edmonds algorithm and Lipson's fraction-free back substitution formula solves $Ax = b$: Note that the divisions by $B_{k,k}$ and by D_i in Algorithm 1 are exact in $\mathbb{Z}[y_1, y_2, \ldots, y_m]$ and $B_{k,k}$ is the determinant of the principal $k \times k$ submatrix of A. However, there is an expression swell. At the last major step of triangularizing B when $k = n - 1$ where it computes

$$B_{n,n} = \frac{B_{n-1,n-1}B_{n,n} \; - \; B_{n,n-1}B_{n-1,n}}{B_{n-2,n-2}} = \det(A), \tag{1.5}$$

the numerator polynomial in (1.5) is the product of determinants $B_{n,n}$ and $B_{n-2,n-2}$ which are polynomials in $\mathbb{Z}[y_1, y_2, \ldots, y_m]$. If the original entries $B_{i,j}$ from B are sparse polynomials in many parameters then the numerator polynomial in (1.5) may be 100 times or more larger than $\det(A)$. The same situation also holds for the polynomials $\tilde{x}_i$.

One approach to avoid this expression swell was tried by Monagan and Vrbik in [**?**]. They compute the quotients of (1.3) and (1.4) directly using lazy polynomial arithmetic without constructing the numerator polynomial in (1.5) in expanded form.

Another approach is to interpolate the polynomials $\tilde{x}_i$ and $\det(A)$ directly from points using sparse polynomial interpolation algorithms [**?**,**?**,**?**,**?**], and Chinese remaindering when needed. This approach is described briefly as follows.

- Pick an evaluation point $\alpha \in \mathbb{Z}_p^m$ and solve

$$A(\alpha)\, x(\alpha) = b(\alpha) \pmod{p}$$

  for $x(\alpha)$ using Gaussian elimination over $\mathbb{Z}_p$ and also compute $\det(A(\alpha))$ at the same time.

- Provided $\det(A(\alpha)) \neq 0$, then

$$\tilde{x}_i(\alpha) = x_i(\alpha) \times \det(A(\alpha)).$$

  Thus, we have images of $\tilde{x}_i$ and $\det(A)$ so we can interpolate them.

To compute the solution vector x in the simplest terms, we compute

$$h_i = \gcd(\tilde{x}_i, \det(A))$$

for $1 \leq i \leq n$ and cancel them from

$$\frac{\tilde{x}_i}{\det(A)}$$

to simplify the solutions. However, in practice there may be a large cancellation in

$$\frac{\tilde{x}_i}{\det(A)}.$$

That is, h_i may be a large factor so that the final solution

$$x_i = \frac{\tilde{x}_i / h_i}{\det(A)/h_i}$$

may be small. Our new algorithm will interpolate x_i directly, thus avoiding any GCD computations which may be expensive. To illustrate the gain realized by our new algorithm for solving $Ax = b$, we give the following real example.

Chapter 2

Maximal Quotient Rational Function Reconstruction

2.1 Univariate rational function reconstruction

Let $\mathbb{F}$ be a field and and $\mathcal{B}$ be a black box for a rational function $h(x) = \frac{f(x)}{g(x)} \in \mathbb{F}(x)$ where $f(x), g(x) \in \mathbb{F}[x]$ and $g(x) \neq 0$ and $\gcd(f, g) = 1$. We need to recover the polynomials $f(x), g(x)$ upto a constant factor. We can evaluate the black box at n distinct points $\alpha_1, \ldots, \alpha_n$ to gets the values $y_1, \ldots, y_n$.

2.2 The extended Euclidean algorithm

Extended Euclidean Algorithm

Input: Polynomials $f, g \in \mathbb{F}[x]$.
Output: $r_l, s_l, t_l \in \mathbb{F}[x]$ such that $r_l = \gcd(f, g) = s_l \cdot f + t_l \cdot g$.
$r_0 \leftarrow f, \quad s_0 \leftarrow 1, \quad t_0 \leftarrow 0$
$r_1 \leftarrow g, \quad s_1 \leftarrow 0, \quad t_1 \leftarrow 1$
$i \leftarrow 1$
while $r_i \neq 0$ **do**
    $q_i \leftarrow r_{i-1}/r_i$ ▷ Quotient
    $r_{i+1} \leftarrow r_{i-1} - q_i r_i$ ▷ Remainder
    $s_{i+1} \leftarrow s_{i-1} - q_i s_i$
    $t_{i+1} \leftarrow t_{i-1} - q_i t_i$
    $i \leftarrow i + 1$
end while
$l \leftarrow i - 1$
return $r_l, s_l, t_l \in \mathbb{F}[x]$

Theorem 2.2.1 (Bezout's identity)**.** *Let $f, g \in \mathbb{F}[x]$ such that $f, g \neq 0$,let $d = gcd(f, g)$. Then $\exists s, t \in \mathbb{F}[x]$ such that,*

$$s_i f + t_i g = d$$

Computing inverse using Bezout's identity

$$\textit{when } gcd(f, g) = 1 \implies s_l f + t_l g = 1$$

$$\implies t_l g \equiv 1 \mod f \implies t_l = \frac{1}{g} \mod f$$

2.3 The Chinese remainder theorem

Theorem 2.3.1. *[3] Let $I_1, I_2, .., I_k$ be ideals in a ring R. The map*

$$\begin{array}{rccl} \phi: & R & \longrightarrow & R/I_1 \times R/I_2 \times \cdots \times R/I_k \\ & r & \longmapsto & (r + I_1, r + I_2, \ldots, r + I_k) \end{array}$$

is a ring homomorphism with kernel $I_1 \cap I_2 \cap \cdots \cap I_k$. If for each $i, j \in \{1, 2, \ldots, k\}$ with $i \neq j$,the ideals I_i, I_j are called comaximal and the map ϕ is surjective. Moreover,

$$I_1 \cap I_2 \cap \cdots \cap I_k = I_1 I_2 \ldots I_k$$

$$\implies R/(I_1 I_2 \ldots I_k) = R/(I_1 \cap I_2 \cap \cdots \cap I_k) \cong R/I_1 \times R/I_2 \times \cdots \times R/I_k.$$

2.4 Rational function reconstruction

Chinese remainder theorem and Interpolation

Theorem 2.4.1. *Let $\mathbb{F}[x]$ be a polynomial ring over a field $\mathbb{F}$ and let $I \subset \mathbb{F}[x]$ be an ideal of the ring then $\mathbb{F}[x]/I$ is a field if and only if I is a maximal ideal.*

Given a black box B for a polynomial $f(x) \in \mathbb{F}[x]$ and $x = \alpha \in \mathbb{F}$ we can evaluate $f(\alpha)$ by querying the black box B at α.

$$\begin{array}{rccl} \phi: & \mathbb{F}[x] & \longrightarrow & \mathbb{F}[x]/(x - \alpha) \\ & f(x) & \longmapsto & f(\alpha) \end{array}$$

Thus, evaluating the polynomial at n points is querying the black box at n points $\alpha_1, \ldots, \alpha_n$ is the following homomorphism.

$$\begin{array}{rccl} \phi: & \mathbb{F}[x] & \longrightarrow & \prod_{i=1}^{n} \mathbb{F}[x]/(x - \alpha_i) \\ & f(x) & \longmapsto & (f(\alpha_1), f(\alpha_2), \ldots, f(\alpha_n)) \end{array}$$

From the Chinese remainder theorem, we know that

$$\prod_{i=1}^{n} \mathbb{F}[x]/(x-\alpha_i) \cong \mathbb{F}[x]/\prod_{i=1}^{n}(x-\alpha_i)$$

$$\text{Let } \alpha \in \mathbb{F}^n$$

such that

$$\alpha_i \neq \alpha_j \forall i \neq j \text{ and } n > 0.$$

$$\text{Let } h(x) = \frac{f(x)}{g(x)} \in \mathbb{F}(x), \text{ where } f(x), g(x) \in \mathbb{F}[x],$$

$$\text{Let } y_i = F(\alpha_i) = \frac{f(\alpha_i)}{g(\alpha_i)}, \forall 1 \leq i \leq n, \text{ such that } g(\alpha_i) \neq 0.$$

$$\exists!\ u(x) \in \mathbb{F}[x] \text{ of degree} < n \text{ such that } u(\alpha_i) = y_i$$

$$\implies u(x) \equiv y_i \mod (x-\alpha_i)$$

$$\implies h(x) = u(x) \equiv \frac{f(x)}{g(x)} \mod (x-\alpha_i) \tag{2.1}$$

$$\implies u(x)g(x) \equiv f(x) \bmod (x-\alpha_i) \tag{2.2}$$

$$\text{Let } \overline{m}(x) = \prod_{i=1}^{n}(x-\alpha_i)$$

From Chinese Remainder theorem,

$$\prod_{i=1}^{n} \mathbb{F}[x]/(x-\alpha_i) \cong \mathbb{F}[x]/\prod_{i=1}^{n}(x-\alpha_i) \implies \prod_{i=1}^{n} \cong \mathbb{F}[x]/\overline{m}(x)$$

$$\implies u(x)g(x) \equiv f(x) \bmod \overline{m}(x) \tag{2.3}$$

We can get the above equation by applying the extended Euclidean algorithm $\overline{m}(x)$ and $u(x)$.
By Bezout's identitiy, we have

$$\overline{m}(x)s(x) + u(x)t(x) = r(x) \tag{2.4}$$

where $r(x) = gcd(u(x), m(x))$ Taking mod $\overline{m}(x)$ leads to the above equation,

$$u(x)t(x) \equiv r(x) \bmod \overline{m}(x)$$

$$\implies u(x) = \frac{r(x)}{t(x)} \bmod \overline{m}(x, \tag{2.5}$$

provided $gcd(t, u) = 1$. Thus, we can use the extended Euclidean algorithm to recover the numerator and the denominator from $u(x)$ and $\overline{m}(x)$.

2.5 Maximal quotient rational function reconstruction

Given the black box $\mathcal{B}$ of a rational function $h(x) \in \mathbb{F}(x)$. We can choose n distinct random points $\alpha_1, \ldots, \alpha_n \in \mathbb{F}$ such that

$$n > \text{ degree of numerator+ degree of denominator}$$

and evaluate the black box $\mathcal{B}$ at these points to get $y_1, \ldots, y_n$.
We then construct $\overline{m}(x) = \prod_{i=1}^{n}(x - \alpha_i)$ and $u(x) = Interpolation(\alpha_i, y_i)$, I am using Maple's implementation of Newton's interpolation to compute $u(x)$. We then apply the extended Euclidean algorithm to the determined $\overline{m}(x)$ and $u(x)$, which will yield the numerator and denominator as $r_i(x)$ and $t_i(x)$ for some iteration i of the extended Euclidean algorithm.However, since there doesn't exist a unique representation of rational functions how do we know which $r_i(x)$ and $t_i(x)$ correspond to the actual numerator and the denominator of the Black Box.
Michael Monagan [14] observed the output of extended Euclidean algorithm and identified that the iteration i that yields the numerator and the denominator can be identified by keeping track of the degree of the quotient produced in each iteration. He observed that $r_i(x)$ and $t_i(x)$ associated with the iteration for which the degree of the quotient is maximum,are the numerator and denominator of out target rational function.

Maximal Quotient Rational Function Reconstruction

The

1: **Input:** $\bar{m}, u \in \mathbb{F}[x]$ where $\mathbb{F}$ is a field and $\deg(m) > \deg(u) \geq 0$ or $u = 0$ and $\deg(\bar{m}) \geq 1$.

2: **Output:** Either $f, g \in \mathbb{F}[x]$ satisfying

$$\frac{f}{g} \equiv u \pmod{\bar{m}}, \quad \gcd(u, g) = \gcd(f, g) = 1, \quad \text{and } \deg(f)+\deg(g)+1 < \deg(\bar{m}),$$

or **FAIL** if no such solution exists.
Remark: The degree requirement $\deg(f)+\deg(g)+1 < \deg(m)$ is met by requiring that one of the quotients q_i in the Euclidean algorithm has degree at least 2.

3: **if** $u = 0$ **then**

4: **return** $(f, g) \leftarrow (0, 1)$
5: **end if**
6: $(r_0, r_1) \leftarrow (\bar{m}, u), \quad (t_0, t_1) \leftarrow (0, 1)$
7: $(f, g) \leftarrow (r_1, t_1), \quad q_{\max} \leftarrow 1$
8: $i \leftarrow 1$
9: **while** $r_i \neq 0$ **do**
10: $q_i \leftarrow r_{i-1}$ quo r_i
11: **if** $\deg(q_i) > q_{\max}$ **then**
12: $q_{\max} \leftarrow \deg(q_i)$
13: $(f, g) \leftarrow (r_i, t_i)$
14: **end if**
15: $(r_{i+1}, t_{i+1}) \leftarrow (r_{i-1} - q_i r_i,\ t_{i-1} - q_i t_i)$
16: $i \leftarrow i + 1$
17: **end while**
18: **if** $q_{\max} \leq 1$ **or** $\gcd(f, g) \neq 1$ **then**
19: **return FAIL**
20: **end if**
21: **return** $(f/\operatorname{lc}(g),\ g/\operatorname{lc}(g))$

Example 2. *Let $\mathbb{F} = \mathbb{Z}_{19}$ and let $\mathcal{B}$ be the blackbox for the rational function*

$$T(x) = \frac{ff(x)}{gg(x)} = \frac{3x^2 + 1}{x + 7} \in \mathbb{Z}_{19}(x)$$

We want to recover the rational function $T(x)$

$$\alpha_1 = 1, \alpha_2 = 2, \alpha_3 = 3, \alpha_4 = 4, \alpha_5 = 5$$

$$\overline{m} = (x-1)(x-2)(x-3)(x-4)(x-5) = x^5 + 4x^4 + 9x^3 + 3x^2 + 8x + 13,$$

$$\mathcal{B}(1) = 10, \mathcal{B}(2) = 12, \mathcal{B}(3) = 18, \mathcal{B}(4) = 1, \mathcal{B}(5) = 0$$

and the interpolating polynomial

$$u = 17x^4 + 6x^3 + 16x^2 + 18x + 10.$$

The following table shows the values of q_i, r_i, t_i when the MQRFR is called with inputs $\overline{m}$ and u.

In row 2 of the table above, the degree of the quotient q_2 is maximal when compared to the degree of other q_i's. So we get

$$\frac{r_2}{t_2} = \frac{11x^2 + 10}{10x + 13}, \qquad (d = 5 > \deg(f) + \deg(g) = 3).$$

Table 2.1: Extended Euclidean algorithm computations for input polynomials $\overline{m}$ and u (over $\mathbb{Z}_{19}$).

i	q_i	r_i	t_i
0	–	$x^5 + 4x^4 + 9x^3 + 3x^2 + 8x + 13$	0
1	$9x + 6$	$17x^4 + 6x^3 + 16x^2 + 18x + 10$	1
2	$5x^2 + 4x + 9$	$11x^2 + 10$	$10x + 13$
3	$9x + 7$	$16x + 15$	$7x^3 + 9x^2 + 10x + 17$
4	–	0	$13x^4 + 3x^3 + 18x^2 + 15x + 8$

Making gg monic,

$$T(x) = \frac{ff(x)}{gg(x)} = \frac{3x^2 + 1}{x + 7}.$$

2.6 Failure probability

By Schwartz-Zipel lemma $Pr[failure] = \frac{\deg g}{p} = \frac{\deg g}{2^{31}-1}$

Chapter 3

Ben-Or Tiwari Interpolation

3.1 Introduction

Ben-Or Tiwari is a two-step multivariate interpolation algorithm that interpolates all the variables of the polynomial simultaneously as opposed to Zippel's multivariate interpolation algorithm, which interpolates the variables one at a time. In the first step, the Ben-Or Tiwari algorithm finds the monomials of the multivariate polynomial and in the second step, it finds the coefficients of the polynomial. Each step of the Ben-Or Tiwari algorithm is based on considering polynomials in different contexts.
First step, uses insights from linear recursive sequences and uses Berlekamp Massey algorithm to recover the monomials, while the second step uses Vandermonde Solver to recover the coefficients.

3.2 Linear recursive sequences

Definition 3.2.1 (Linear recurrence sequence)**.** *Any sequence* $\{a_n\}$*, satisfying a linear homogeneous recurrence with constant coefficients of the form*

$$a_n = c_0 a_{n-k} + c_1 a_{n-k+1} + \cdots + c_{k-1} a_{n-1} \ \forall \ n \geq k$$

is called a linear recursive sequence. Where $c_0, c_1, \ldots, c_{k-1}$ *are constants, and* k *is the order of the recurrence.*

3.2.1 Linear recurrence as a linear system

Linear recurrences have a rich structure that can be explored using concepts from linear algebra. We will see two different way to express linear recurrence as linear systems,

1. Using Companion matrix, lets us explore the structure of linear recurrences.
2. Using Hankel matrix, lets us understand how they can useful for Ben-Or Tiwari interpolation.

The following theorem establishes that vector space of all sequences satisfying a linear recurrence is isomorphic to a vector space.

Theorem 3.2.1. *[16]*
Given $c_0, c_1, \ldots, c_{k-1} \in \mathbb{F}$, let

$$V = \{(a_n) \mid a_n = c_0 a_{n-k} + c_1 a_{n-k+1} + \cdots + c_{k-1} a_{n-1}, \forall\ n \geq k\}$$

denote the vector space of all sequences satisfying the linear recurrence relation determined by $c_0, c_1, \ldots, c_{k-1}$. Then the function

$$T : \mathbb{F}^k \to V$$

defined above is an isomorphism. In particular:

1. $\dim V = k$.
2. *If $\{\mathbf{v}_1, \ldots, \mathbf{v}_k\}$ is any basis of $\mathbb{F}^k$, then $\{T(\mathbf{v}_1), \ldots, T(\mathbf{v}_k)\}$ is a basis of V.*

Now that we have the isomorphism with vector spaces, we can think of linear recurrences in terms of linear transforms and see what the structure and properties of the matrix of linear transform tell us.

Companion matrix

Consider a linear homogeneous recurrence relation with constant coefficients of order k:

$$a_n = c_0 a_{n-k} + c_1 a_{n-k+1} + \cdots + c_{k-1} a_{n-1}$$

We can rewrite this as the following linear transform.

$$C = \begin{bmatrix} 0 & 1 & 0 & \cdots & 0 \\ 0 & 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & 1 \\ c_0 & c_1 & c_2 & \cdots & c_{k-1} \end{bmatrix} \begin{bmatrix} a_{n-k} \\ a_{n-k+1} \\ \vdots \\ a_{n-2} \\ a_{n-1} \end{bmatrix} = \begin{bmatrix} a_{n-k+1} \\ a_{n-k+2} \\ \vdots \\ a_{n-1} \\ c_0 a_{n-k} + c_1 a_{n-k+1} + \cdots + c_{k-1} a_{n-k} \end{bmatrix}$$

where C is called the **companion matrix**.
This setup looks similar to a dynamic system, if we define the inital state of the system as $\mathbf{v}_0$ with the initial values of the sequence, then $\mathbf{v}_0 = [a_0, a_1, \ldots, a_{k-1}]^T$ Then we can define the state of the system at step i as

$$v_i = C^i v_0$$

Where $\mathbf{v}_i = [a_i, a_{i+1}, \ldots, a_{i+k-1}]^T$. The recurrence can be written in matrix form:

$$\mathbf{v}_{i+1} = C\mathbf{v}_i,$$

3.2.2 Characteristic polynomial of the companion matrix

Claim 1. *The characteristic polynomial of the companion matrix*

$$C = \begin{bmatrix} 0 & 1 & 0 & \cdots & 0 \\ 0 & 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & 1 \\ c_0 & c_1 & c_2 & \cdots & c_{k-1} \end{bmatrix}_{k\times k}$$

is given by

$$\lambda^k - c_{k-1}\lambda^{k-1} - c_{k-2}\lambda^{k-2} - \cdots - c_1\lambda - c_0$$

Proof.

$$M_k(\lambda) := \lambda I - C = \begin{bmatrix} \lambda & -1 & 0 & \cdots & 0 \\ 0 & \lambda & -1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \ldots & -1 \\ -c_0 & -c_1 & -c_2 & \cdots & \lambda - c_{k-1} \end{bmatrix}$$

Expanding along the first column Column 1 has only two nonzero entries, so

$$\Delta_k(\lambda) = \lambda \det M_k^{(1,1)} \; + \; (-c_0)(-1)^{k+1} \det M_k^{(k,1)},$$

where $M_k^{(i,j)}$ denotes the minor formed by deleting row i and column j.

The $(k,1)$-minor. Deleting row k and column 1 leaves a lower–bidiagonal matrix with diagonal entries -1:

$$M_k^{(k,1)} = \begin{bmatrix} -1 & 0 & \cdots & 0 \\ \lambda & -1 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & -1 \end{bmatrix}, \quad \Rightarrow \quad \det M_k^{(k,1)} = (-1)^{k-1}.$$

Hence the second term equals

$$(-c_0)(-1)^{k+1}(-1)^{k-1} = -c_0.$$

The $(1,1)$-minor. Deleting row 1, column 1 yields

$$M_k^{(1,1)} = \begin{bmatrix} \lambda & -1 & 0 & \cdots & 0 \\ 0 & \lambda & -1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & -1 \\ -c_1 & -c_2 & -c_3 & \cdots & \lambda - c_{k-1} \end{bmatrix}.$$

We now evaluate $\det M_k^{(1,1)}$ inductively.

Using Induction to evaluate $\det M_k^{(1,1)}$

For $m \geq 1$, define

$$B_m(\lambda) = \begin{bmatrix} \lambda & -1 & 0 & \cdots & 0 \\ 0 & \lambda & -1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & -1 \\ -d_1 & -d_2 & -d_3 & \cdots & \lambda - d_m \end{bmatrix}.$$

Claim. $\det B_m(\lambda) = \lambda^m - d_m\lambda^{m-1} - d_{m-1}\lambda^{m-2} - \cdots - d_2\lambda - d_1$.

Proof by induction.

Base case: $m = 1$: $\det[\lambda - d_1] = \lambda - d_1$ (true).

Inductive case: Assume the formula holds for $m-1$. Expanding $\det B_m$ along the first column:

$$\det B_m(\lambda) = \lambda \det B_{m-1}(\lambda) + (-d_1)(-1)^{m+1} \det T_{m-1},$$

where

$$B_{m-1} = \begin{bmatrix} \lambda & -1 & 0 & \cdots & 0 & 0 \\ 0 & \lambda & -1 & \ddots & 0 & 0 \\ \vdots & \vdots & \vdots & \cdots & \vdots & \vdots \\ 0 & 0 & 0 & \cdots & \lambda & -1 \\ -d_2 & -d_3 & -d_4 & \cdots & -d_{m-1} & \lambda - d_m \end{bmatrix} \text{ and } T_{m-1} = \begin{bmatrix} -1 & 0 & 0 & \cdots & 0 & 0 \\ \lambda & -1 & 0 & \cdots & 0 & 0 \\ \vdots & \vdots & \vdots & \cdots & \vdots & \vdots \\ 0 & 0 & 0 & \cdots & -1 & 0 \\ 0 & 0 & 0 & \cdots & \lambda & -1 \end{bmatrix}$$

T_{m-1} is lower–bidiagonal with diagonal entries -1, hence $\det T_{m-1} = (-1)^{m-1}$. Thus the second term is $(-d_1)(-1)^{m+1}(-1)^{m-1} = -d_1$.

By the induction hypothesis,

$$\det B_{m-1}(\lambda) = \lambda^{m-1} - d_m\lambda^{m-2} - \cdots - d_3\lambda - d_2$$

$$\det B_m(\lambda) = \lambda(\lambda^{m-1} - d_m\lambda^{m-2} - \cdots - d_3\lambda - d_2) - d_1$$

$$\det B_m(\lambda) = \lambda^m - d_m\lambda^{m-1} - \cdots - d_3\lambda^2 - d_2\lambda - d_1$$

Applying the claim with $m = k-1$ for $(d_1, \ldots, d_{k-1}) = (c_1, \ldots, c_{k-1})$ gives us

$$\det M_k^{(1,1)} = \lambda^{k-1} - c_{k-1}\lambda^{k-2} - \cdots - c_2\lambda - c_1.$$

Substituting $M_k^{(1,1)}$ in $\Delta_k(\lambda) = \lambda \det M_k^{(1,1)} + (-c_0)(-1)^{k+1} \det M_k^{(k,1)}$

$$\begin{aligned}\Delta_k(\lambda) &= \lambda(\lambda^{k-1} - c_{k-1}\lambda^{k-2} - \cdots - c_2\lambda - c_1) - c_0 \\ &= \lambda^k - c_{k-1}\lambda^{k-1} - c_{k-2}\lambda^{k-2} - \cdots - c_1\lambda - c_0\end{aligned}$$

□

A general solution is a basis that generates each element of the recurrence relation as a linear combination.

3.2.3 Eigen values of the companion matrix are the general solution of the linear recurrence

Given a linear homogeneous linear recurrence relation with constant coefficients of order k:

$$a_n = c_0 a_{n-k} + c_1 a_{n-k+1} + \cdots + c_{k-1} a_{n-1}$$

The characteristic polynomial $\Lambda(\lambda)$ of the companion matrix C is given by

$$\lambda^k - c_{k-1}\lambda^{k-1} - c_{k-2}\lambda^{k-2} - \cdots - c_1\lambda - c_0 = 0$$

Let the characteristic polynomial have k distinct roots $\lambda_1, \ldots, \lambda_k$, then since C is a square matrix with distinct eigen values, C is diagonizable,

$$C = PDP^{-1} \text{ Where } D = diag(\lambda_1, \lambda_2, \cdots, \lambda_k)$$

We saw earlier that we can formulate linear recurrence as the following dynamic system.

$$v_i = C^i v_0 \implies v_i = (PD^iP^{-1})v_0$$

Where $D^i = diag(\lambda_1^i, \lambda_2^i, \cdots, \lambda_k^i)$, and $\mathbf{v}_i = [a_i, a_{i+1}, \ldots, a_{i+k-1}]^T$. Thus, we can see that the companion matrix acts as a shift operator for the input vector v_0.

Constructing matrix P

Let the characteristic polynomial of the companion matrix C of rank k have k distinct roots $\lambda_1, \ldots, \lambda_k$.

$$w = \begin{bmatrix} w_0 \\ w_1 \\ \vdots \\ w_{k-1} \end{bmatrix} \text{ be an eigen vector of } C \implies Cw = \lambda w$$

Since the companion matrix acts as a shift operator, we have

$$Cw = \begin{bmatrix} 0 & 1 & 0 & \cdots & 0 \\ 0 & 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & 1 \\ c_0 & c_1 & c_2 & \cdots & c_{k-1} \end{bmatrix} \begin{bmatrix} w_0 \\ w_1 \\ \vdots \\ w_{k-2} \\ w_{k-1} \end{bmatrix} = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_{k-1} \\ c_0 w_0 + c_1 w_1 + \cdots + c_{k-1} w_{k-1} \end{bmatrix} = \begin{bmatrix} \lambda w_0 \\ \lambda w_1 \\ \vdots \\ \lambda w_{k-2} \\ \lambda w_{k-1} \end{bmatrix}$$

For the first $k-1$ entries of Cw we can see that

$$w_{i+1} = \lambda w_i$$

$$\implies w_1 = \lambda w_0,\ w_2 = \lambda w_1 = \lambda^2 w_0,\ w_3 = \lambda^3 w_0, \ldots, w_{k-1} = \lambda^{k-1} w_0$$

For the last entry of Cw, we have

$$c_0 w_0 + c_1 w_1 + \cdots + c_{k-1} w_{k-1} = \lambda w_{k-1}$$

Substituting $w_i = \lambda^i w_0$ in above equation,

$$c_0 w_0 + c_1 \lambda w_0 + \cdots + c_{k-1} \lambda^{k-1} w_0 = \lambda(\lambda^{k-1} w_0) = \lambda^k w_0$$

Dividing both sides by w_0, gives us the characteristic polynomial

$$\lambda^k - c_{k-1}\lambda^{k-1} - c_{k-2}\lambda^{k-2} - \cdots - c_1\lambda - c_0 = 0$$

$$\implies w_0 = 1,\ w_1 = \lambda,\ w_2 = \lambda^2, \ldots,\ w_{k-1} = \lambda^{k-1},$$

Thus, the eigenvector

$$w(\lambda) = \begin{bmatrix} 1 \\ \lambda \\ \lambda^2 \\ \vdots \\ \lambda^{k-1} \end{bmatrix}$$

Diagonalization matrix P

$$P = \begin{bmatrix} w(\lambda_1) & \cdots & w(\lambda_k) \end{bmatrix} = \begin{bmatrix} 1 & \cdots & 1 \\ \lambda_1 & \cdots & \lambda_k \\ \vdots & \ddots & \vdots \\ \lambda_1^{k-1} & \cdots & \lambda_k^{k-1} \end{bmatrix}, \quad C = P \operatorname{diag}(\lambda_1, \ldots, \lambda_k) P^{-1}.$$

3.2.4 Hankel matrix

What if we don't know the linear recurrence, but we can generate infinitely many terms of the sequence. How to find the characteristic polynomial in this case? For this we need Hankel Matrix.

Definition 3.2.2. *Given a sequence $\{a_n\}$ the corresponding Hankel matrix is given by*

$$H_m = \begin{bmatrix} a_0 & a_1 & a_2 & \ldots & a_{m-1} \\ a_1 & a_2 & a_3 & \ldots & a_m \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_{m-1} & a_m & a_{m+1} & \cdots & a_{2m-2} \end{bmatrix}_{m \times m}$$

The Hankel matrix H_m can be an infinite matrix if use infinitely many terms of the sequence to construct it.

Theorem 3.2.2 (Kronecker-Hankel Theorem [6])**.** *A sequence of numbers satisfies a linear recurrence relation if and only if the associated infinite Hankel matrix has finite rank. The order of the linear recurrence is equal to the rank of the Hankel matrix.*

Consequence of Kronecker-Hankel theorem

Let $\{a_n\}$ be a homogeneous linear recurrence of order k such that

$$a_n = c_0 a_{n-k} + c_1 a_{n-k+1} + \cdots + c_{k-1} a_{n-1}$$

Let H_m be the infinite Hankel matrix constructed from the terms of $\{a_n\}$, then H_k will have rank k and $\exists$ a submatrix H_k such that

$$H_k = \begin{bmatrix} v_0^T \\ [Cv_0]^T \\ \vdots \\ [C^{k-1}v_0]^T \end{bmatrix} \text{ where } v_0 = \begin{bmatrix} a_0 \\ a_1 \\ \vdots \\ a_{k-1} \end{bmatrix} \text{ and } C = \begin{bmatrix} 0 & 1 & 0 & \cdots & 0 \\ 0 & 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & 1 \\ c_0 & c_1 & c_2 & \cdots & c_{k-1} \end{bmatrix}$$

Gaussian Elimination of Hankel Matrix

$$H_k\bar{c} = \begin{bmatrix} a_0 & a_1 & \cdots & a_{k-1} \\ a_1 & a_2 & \cdots & a_k \\ \vdots & \vdots & \ddots & \vdots \\ a_{k-1} & a_k & \cdots & a_{2k-1} \end{bmatrix} \begin{bmatrix} c_0 \\ c_1 \\ \vdots \\ c_{k-1} \end{bmatrix} = \begin{bmatrix} c_0a_0 + c_1a_1 + \cdots + c_{k-1}a_{k-1} \\ c_0a_1 + c_1a_2 + \cdots + c_{k-1}a_k \\ \vdots \\ c_0a_{k-1} + c_1a_k + \cdots + c_{k-1}a_{2k-1} \end{bmatrix} = \begin{bmatrix} a_k \\ a_{k+1} \\ \vdots \\ a_{2k} \end{bmatrix}$$

We can solve the linear system above using Gaussian elimination in $O(k^3)$ to find the coefficients of the linear recurrence which gives us the minimal characteristic polynomial

$$\lambda^k - c_{k-1}\lambda^{k-1} - c_{k-2}\lambda^{k-2} - \cdots - c_1\lambda - c_0 = 0$$

3.2.5 Polynomials as linear recurrences.

Lemma 3.2.3. *Let $\mathbb{F}[x]$ be a ring of polynomials over field $\mathbb{F}$, let $f(x) \in \mathbb{F}[x]$, and $\alpha \in \mathbb{F}$ be some arbitrary element of $\mathbb{F}$, then the sequence*

$$\{f(1), f(\alpha), f(\alpha^2), \ldots, f(\alpha^n)\}$$

is a linear recursive sequence.

Proof. Let

$$f(x) = b_{k-1}x^{k-1} + b_{k-2}x^{k-2} + \cdots + b_1x + b_0 \in \mathbb{F}[x],$$

where $\mathbb{F}$ is a field. For some arbitrary $\alpha \in \mathbb{F}$, we define the sequence $\{a_n\}$ such that

$$a_i = f(\alpha^i) = \sum_{j=0}^{k-1} b_j(\alpha^i)^j = \sum_{j=0}^{k-1} b_j(\alpha^j)^i.$$

Hence, each a_i is a linear combination of the geometric sequences $(\alpha^j)^i$ for $j = 0, 1, \ldots, k-1$.

The corresponding $m \times m$ Hankel matrix associated with the sequence $\{a_i\}$ is

$$H_m = \begin{bmatrix} a_0 & a_1 & a_2 & \cdots & a_{m-1} \\ a_1 & a_2 & a_3 & \cdots & a_m \\ a_2 & a_3 & a_4 & \cdots & a_{m+1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_{m-1} & a_m & a_{m+1} & \cdots & a_{2m-2} \end{bmatrix}.$$

Using Rank decomposition, we can factorize the Hankel matrix as follows [2],

$$H_m = V_m D V_m^\top.$$

Where,

1. Vandermonde-type matrix V_m:

$$V_m = \begin{bmatrix} 1 & 1 & 1 & \cdots & 1 \\ \alpha^0 & \alpha^1 & \alpha^2 & \cdots & \alpha^{k-1} \\ (\alpha^0)^2 & (\alpha^1)^2 & (\alpha^2)^2 & \cdots & (\alpha^{k-1})^2 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ (\alpha^0)^{m-1} & (\alpha^1)^{m-1} & (\alpha^2)^{m-1} & \cdots & (\alpha^{k-1})^{m-1} \end{bmatrix}_{m \times k}.$$

2. Diagonal matrix D:

$$D = \operatorname{diag}(b_0, b_1, \ldots, b_{k-1}) = \begin{bmatrix} b_0 & 0 & 0 & \cdots & 0 \\ 0 & b_1 & 0 & \cdots & 0 \\ 0 & 0 & b_2 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & b_{k-1} \end{bmatrix}_{k \times k}.$$

3. Transpose $V_m^\top$:

$$V_m^\top = \begin{bmatrix} 1 & \alpha^0 & (\alpha^0)^2 & \cdots & (\alpha^0)^{m-1} \\ 1 & \alpha^1 & (\alpha^1)^2 & \cdots & (\alpha^1)^{m-1} \\ 1 & \alpha^2 & (\alpha^2)^2 & \cdots & (\alpha^2)^{m-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \alpha^{k-1} & (\alpha^{k-1})^2 & \cdots & (\alpha^{k-1})^{m-1} \end{bmatrix}_{k \times m}.$$

Each entry of H_m satisfies

$$a_{p+q} = \sum_{j=0}^{k-1} b_j(\alpha^j)^{p+q} = \sum_{j=0}^{k-1} b_j(\alpha^j)^p(\alpha^j)^q.$$

The (p,q)-entry of $V_m D V_m^\top$ is

$$(V_m D V_m^\top)_{p,q} = \sum_{j=0}^{k-1}(V_m)_{p,j} D_{j,j}(V_m^\top)_{j,q} = \sum_{j=0}^{k-1} b_j(\alpha^j)^p(\alpha^j)^q = a_{p+q}.$$

Thus, $H_m = V_m D V_m^\top$.

Since V_m is an $m \times k$ matrix, we have

$$\operatorname{rank}(V_m) \leq k,$$

$$\implies \operatorname{rank}(H_m) = \operatorname{rank}(V_m D V_m^\top) \leq \operatorname{rank}(V_m) \leq k.$$

Now for $m = k+1$,

$$\operatorname{rank}(H_{k+1}) \leq k < k+1,$$

so

$$\det(H_{k+1}) = 0$$

By theorem 3.2.2 we can say that the entries of the matrix H_k, the terms of the sequence $\{f(\alpha^i)\}$ form a linear recurrence.

□

The above argument can be extended to multivariate polynomials as well.
Let $f(x_1, x_2, \cdots, x_n) \in \mathbb{Z}_p[x_1, x_2, \cdots, x_n]$ with t terms

$$f(x_1, \ldots, x_n) = \sum_{j=1}^{t} c_j x_1^{e_j^1} \cdots x_n^{e_j^n} = \sum_{j=1}^{t} c_j \mathcal{M}_j(x_1, \ldots, x_n), \qquad c_j \neq 0. \tag{3.1}$$

Where $\mathcal{M}_j(x_1, \ldots, x_n)$ are the monomials of f. Let $\alpha = \begin{bmatrix}\alpha_1 & \alpha_2 & \cdots & \alpha_n\end{bmatrix}^T \in \mathbb{Z}_p^n$, then

$$f(\alpha_1, \ldots, \alpha_n) = \sum_{j=1}^{t} c_j \alpha_1 e_j^1 \cdots \alpha_n^{e_j^n} = \sum_{j=1}^{t} c_j \mathcal{M}_j(\alpha_1, \ldots, \alpha_n)$$

$$\implies f(\alpha_1,\ldots,\alpha_n) = \begin{bmatrix}\mathcal{M}_1(\alpha_1,\ldots,\alpha_n) & \mathcal{M}_2(\alpha_1,\ldots,\alpha_n) & \cdots & \mathcal{M}_t(\alpha_1,\ldots,\alpha_n)\end{bmatrix}^T \begin{bmatrix}c_1\\c_2\\\vdots\\c_t\end{bmatrix}$$

$$\implies f(\alpha_1^i,\ldots,\alpha_n^i) = \begin{bmatrix}\mathcal{M}_1(\alpha_1^i,\ldots,\alpha_n^i) & \mathcal{M}_2(\alpha_1^i,\ldots,\alpha_n^i) & \cdots & \mathcal{M}_t(\alpha_1^i,\ldots,\alpha_n^i)\end{bmatrix}^T \begin{bmatrix}c_1\\c_2\\\vdots\\c_t\end{bmatrix}$$

Let $\mathcal{M}_j(\alpha_1^i,\ldots,\alpha_n^i) = m_{ij}$

$$\implies f(\alpha_1^i,\ldots,\alpha_n^i) = \begin{bmatrix}m_{i1} & m_{i2} & \cdots & m_{it}\end{bmatrix}^T \begin{bmatrix}c_1\\c_2\\\vdots\\c_t\end{bmatrix}$$

In this case, the following sequence will be a linear recurrence of order t.

$$\{f(1,1,\ldots,1), f(\alpha_1,\ldots,\alpha_n), f(\alpha_1^2,\ldots,\alpha_n^2),\ldots,f(\alpha_1^i,\ldots,\alpha_n^i)\}$$

3.3 Berlekamp Massey algorithm

The Berlekamp Massey algorithm (BMA) was created by Elwyn Berlekamp to decode BCH code and Reed Solomon codes. James Massey applied it to find the shortest linear feedback shift register(LFSR) for a binary sequence. Given $2t$ terms of a linear recurrence sequence over an arbitrary field, the BMA can find minimal characteristic polynomial in $O(n^2)$. This is an improvement of an order of magnitude compared to finding the coefficients of the characteristic polynomial by using Gaussian Elimination of the Hankel matrix.

The BMA iteratively builds the minimal polynomial by processing the sequence $\{s_n\}$ and adjusting for discrepancies

- Initialize $C(x) = 1$, $L = 0$ (degree of the LFSR), and process the sequence s_i.
- For each i, compute the discrepancy:

$$\Delta = s_i + \sum_{j=1}^{L} c_j s_{i-j}.$$

If $\Delta \neq 0$, update $C(x)$ using a previous polynomial and adjust L if necessary.

- The final $C(x)$ is the minimal polynomial $\sigma(x)$.

Berlekamp-Massey Algorithm (BMA)

Input: Sequence $s = \{s_0, s_1, \ldots, s_{n-1}\}$ over field $\mathbb{F}$.
Output: Coefficients of minimal LFSR polynomial $c = \{c_1, c_2, \ldots, c_l\}$ satisfying $s_i = \sum_{j=1}^{l} c_j s_{i-j}$ for $i \geq l$.
$c \leftarrow \{\}$ ▷ Current linear recurrence sequence
oldC $\leftarrow \{\}$ ▷ Best previous version of c
$f \leftarrow -1$ ▷ Index where oldC failed
for $i \leftarrow 0$ **to** $n-1$ **do**
    $\Delta \leftarrow s_i$ ▷ Evaluate $c(i) = \sum_{j=1}^{|c|} c_j s_{i-j}$
    for $j \leftarrow 1$ **to** $|c|$ **do**
        $\Delta \leftarrow \Delta - c_j \cdot s_{i-j}$
    end for
    if $\Delta = 0$ **then**
        continue ▷ $c(i) = s_i$ is correct
    end if
    if $f = -1$ **then** ▷ First time updating c
        $c \leftarrow \{0, 0, \ldots, 0\}$ ▷ Resize to length $i+1$
        $f \leftarrow i$
    else
        $d \leftarrow$ oldC ▷ Build correction sequence
        for $x \in d$ **do** ▷ Step 1: Multiply by -1
            $x \leftarrow -x$
        end for
        d.insert$(0, 1)$ ▷ Step 2: Insert 1 at beginning
        $df_1 \leftarrow 0$ ▷ Step 3: Calculate $d(f+1)$
        for $j \leftarrow 1$ **to** $|d|$ **do**
            $df_1 \leftarrow df_1 + d_j \cdot s_{f+1-j}$
        end for
        coef $\leftarrow \Delta / df_1$
        for $x \in d$ **do** ▷ Step 4: Scale by $\Delta / d(f+1)$
            $x \leftarrow x \cdot$ coef
        end for
        d.insertZeros$(i - f - 1)$ ▷ Step 5: Insert $i - f - 1$ zeros at beginning
        temp $\leftarrow c$ ▷ Save current c
        Resize c to $\max(|c|, |d|)$
        for $j \leftarrow 1$ **to** $|d|$ **do**

```
            c_j ← c_j + d_j
        end for
        if i − |temp| > f − |oldC| then          ▷ Update best previous version
            oldC ← temp
            f ← i
        end if
    end if
end for
return c
```

3.3.1 Equivalence of Berlekamp Massey algorithm and extended Euclidean algorithm

Let $\{s_n\}$ be a sequence with $s_0, s_1, \ldots, s_{2t-1}$, as the first $2t$ terms, the corresponding syndrome polynomial $S(x) = s_0 + s_1 x + \cdots + s_{2t-1}x^{2t-1}$. Decoding error-correcting codes, involve solving the **key equation** [1]:

$$S(x)\sigma(x) \equiv \omega(x) \pmod{x^{2t}},$$

where: $\sigma(x)$ is the error locator polynomial,it is the shortest LFSR (minimal characteristic polynomial) that genrates the syndrome sequence, $\omega(x)$ is the error evaluator polynomial and $2t$ is the error-correcting capability of the code.

The Extended Euclidean Algorithm can solve the key equation by computing:

$$s(x)x^{2t} + t(x)S(x) = r(x),$$

where $r(x)$ is the GCD of x^{2t} and $S(x)$. The algorithm stops when the degree of the remainder $r(x)$ is less than t. At this point, $t(x)$ corresponds to the error locator polynomial $\sigma(x)$, and $r(x)$ corresponds to the error evaluator polynomial $\omega(x)$.

Berlekamp–Massey Euclidean Algorithm (BMEA)

Input: Sequence $S = [s_0, s_1, \ldots, s_{2n-1}] \in \mathbb{Z}_p^{2n}$ of length $2n$, prime number p, indeterminate Z
Output: minimal characteristic polynomial $t_i(Z) \in \mathbb{Z}_p[Z]$ with $\deg(t_i) < n$
$n \leftarrow \text{length}(S)/2$
$m \leftarrow 2n - 1$
$r_0 \leftarrow Z^{2n}$
$r_1 \leftarrow \sum_{i=0}^{2n-1} s_{2n-1-i} \cdot Z^i$ ▷ Computing Syndrome polynomial
$t_0 \leftarrow 0, \quad t_1 \leftarrow 1$

$i \leftarrow 1$
while $\deg(r_i) \geq n$ **do** ▷ EEA
    $q_i \leftarrow r_{i-1}/r_i$
    $r_{i+1} \leftarrow r_{i-1} - q_i r_i$
    $t_{i+1} \leftarrow t_{i-1} - q_i t_i$
    $i \leftarrow i + 1$
end while
Normalize to make t_i Monic:
$\ell \leftarrow \text{leading\_coeff}(t_i)$
$\ell^{-1} \leftarrow \ell^{-1} \bmod p$
return $\ell^{-1} \cdot t_i \bmod p$

3.4 Polynomial evaluation as dot product

Let $f(x) = a_0 + a_1 x + \cdots + a_{n-1}x^{n-1} \in \mathbb{F}[x]$, where $\mathbb{F}[x]$ is a ring of polynomials over the field $\mathbb{F}$. Let $y, \alpha \in \mathbb{F}$ such that $y = f(\alpha)$, then we know that $\mathbb{F}[x]/(x-\alpha) \cong \mathbb{F}$. We can think of polynomial evaluation as the following dot product between the monomial basis at α and the coefficient vector,

$$y = f(\alpha) = a_0 + a_1\alpha + a_2\alpha^2 + \cdots + a_{n-1}\alpha^{n-1} = \begin{bmatrix} 1 & \alpha & \alpha^2 & \cdots & \alpha^{n-1} \end{bmatrix}^T \begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ \vdots \\ a_{n-1} \end{bmatrix}$$

3.5 Polynomial interpolation as a linear transform

Let $\{(\alpha_i, y_i) | 0 \leq i \leq n-1\}$ be a set of n distinct points in $\mathbb{F}^2$. We want to find a unique polynomial $f(x) \in \mathbb{F}[x]$ such that $y_i = f(\alpha_i)$.

$$\text{Let } f(x) = a_0 + a_1 x + \cdots + a_{n-1}x^{n-1}$$

We can write the polynomial evaluation as a dot product as seen in the previous section,

$$\because y_i = f(\alpha_i) \implies y_i = a_0 + a_1\alpha_i + \cdots + a_{n-1}\alpha_i^{n-1} = \begin{bmatrix} 1 & \alpha_i & \alpha_i^2 & \cdots & \alpha_i^{n-1} \end{bmatrix}^T \begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ \vdots \\ a_{n-1} \end{bmatrix}$$

We know from Chinese remainder theorem that evaluating a polynomial $f \in \mathbb{F}[x]$ at n distinct points can be expressed by the following homomorphism,

$$\mathbb{F}[x] \longrightarrow \mathbb{F}[x]/(x-\alpha_0) \times \mathbb{F}[x]/(x-\alpha_1) \times \cdots \times \mathbb{F}[x]/(x-\alpha_{n-1}) \cong \mathbb{F}^n$$

Thus, we can assume $\vec{\alpha}, \vec{y} \in \mathbb{F}^n$ where,

$$\vec{\alpha} = \begin{bmatrix} \alpha_0 \\ \alpha_1 \\ \vdots \\ \alpha_{n-1} \end{bmatrix}, \vec{y} = \begin{bmatrix} y_0 \\ y_1 \\ \vdots \\ y_{n-1} \end{bmatrix}$$

Then can write the we want to find the polynomial f such that $\vec{y} = f(\vec{\alpha})$ as

$$\begin{bmatrix} 1 & \alpha_0 & \dots & \alpha_0^{n-1} \\ 1 & \alpha_1 & \dots & \alpha_1^{n-1} \\ \vdots & \vdots & \dots & \vdots \\ 1 & \alpha_{n-1} & \dots & \alpha_{n-1}^{n-1} \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ \vdots \\ a_{n-1} \end{bmatrix} = \begin{bmatrix} y_0 \\ y_1 \\ \vdots \\ y_{n-1} \end{bmatrix}$$

Where,

$\begin{bmatrix} 1 & \alpha_0 & \dots & \alpha_0^{n-1} \\ 1 & \alpha_1 & \dots & \alpha_1^{n-1} \\ \vdots & \vdots & \dots & \vdots \\ 1 & \alpha_{n-1} & \dots & \alpha_{n-1}^{n-1} \end{bmatrix}$ is the Transposed Vandermonde matrix and $\begin{bmatrix} a_0 \\ a_1 \\ \vdots \\ a_{n-1} \end{bmatrix}$ is the coefficient vector.

Thus, when expressed like this, the problem of polynomial interpolation is transformed to finding a solution the transposed Vandermonde linear system $V\vec{a} = \vec{y}$.

$$\therefore \vec{a} = V^{-1}\vec{y}.$$

However, Solving this system using Gaussian elimination takes $O(n^3)$ time.

3.5.1 Zippel transposed Vandermonde solver

Zippel proposed a method for solving the transposed Vandermonde linear system in $O(n^2)$. The transpose Vandermonde linear system expresses the polynomial in monomial basis. But we can express the polynomial in Lagrangian basis using Lagrange interpolation.

The Lagrange interpolating polynomial is:

$$f(x) = \sum_{i=0}^{n-1} \ell_i(x) y_i = \begin{bmatrix} l_0(x) & l_1(x) & \cdots & l_{n-1}(x) \end{bmatrix}^T \begin{bmatrix} y_0 \\ y_1 \\ \vdots \\ y_{n-1} \end{bmatrix}$$

where the Lagrangian polynomials bases are:

$$\ell_i(x) = \prod_{\substack{j=0 \\ j \neq i}}^{n-1} \frac{x - \alpha_j}{\alpha_i - \alpha_j},$$

We can express the expand each Lagrangian polynomial bases and write it in monomial bases as

$$l_i(x) = c_{0,i} + c_{1,i}x + \cdots + c_{n-1,i}x^{n-1} = \begin{bmatrix} 1 & x & \cdots & x^{n-1} \end{bmatrix}^T \begin{bmatrix} c_{0,i} \\ c_{1,i} \\ \vdots \\ c_{n-1,i} \end{bmatrix}$$

We can express all the Lagrangian bases for the interpolating polynomial as the following linear system,

$$\begin{bmatrix} 1 & x & \cdots & x^{n-1} \end{bmatrix}^T \begin{bmatrix} c_{0,0} & c_{0,1} & \cdots & c_{0,n-1} \\ c_{1,0} & c_{1,1} & \cdots & c_{1,n-1} \\ \vdots & \vdots & \cdots & \vdots \\ c_{n-1,0} & c_{n-1,1} & \cdots & c_{n-1,n-1} \end{bmatrix}$$

Thus, the Lagrangian interpolating polynomial can be expressed as following,

$$f(x) = \begin{bmatrix} 1 & x & \cdots & x^{n-1} \end{bmatrix}^T \begin{bmatrix} c_{0,0} & c_{0,1} & \cdots & c_{0,n-1} \\ c_{1,0} & c_{1,1} & \cdots & c_{1,n-1} \\ \vdots & \vdots & \cdots & \vdots \\ c_{n-1,0} & c_{n-1,1} & \cdots & c_{n-1,n-1} \end{bmatrix} \begin{bmatrix} y_0 \\ y_1 \\ \vdots \\ y_{n-1} \end{bmatrix} = \vec{x} C_L \vec{y}$$

Where, $\vec{x}$ is the monomial basis, C_L is the coefficient matrix for the Lagrange basis.

$$\implies f(x) = \begin{bmatrix} 1 & x & \cdots & x^{n-1} \end{bmatrix}^T \begin{bmatrix} a_0 \\ a_1 \\ \vdots \\ a_{n-1} \end{bmatrix} = \begin{bmatrix} 1 & x & \cdots & x^{n-1} \end{bmatrix}^T C_L \vec{y}$$

$$\implies \vec{a} = \begin{bmatrix} a_0 \\ a_1 \\ \vdots \\ a_{n-1} \end{bmatrix} = C_L \vec{y}$$

We have $\vec{a} = V^{-1}\vec{y}$

$$\implies V^{-1}\vec{y} = C_L\vec{y} \implies V^{-1} = C_L.$$

Calculating the Lagrangian basis takes $O(n^2)$.

The Ben-Or Tiwari algorithm uses the Zippel Vandermonde solver to find the coefficients of the target multivariate polynomial after determining the monomials by using the Cantor-Zassenhaus algorithm to factorize the minimal characteristic polynomial obtained from the Berlekamp Massey algorithm. Thus, we can use $\Lambda(z)$ as the master polynomial $M(x)$ and use it to derive the individual Lagrangian bases.

3.6 Ben-Or and Tiwari's multivariate polynomial interpolation

We have covered all the components of the Ben-Or Tiwari interpolation algorithm, here we describe the algorithm. Let $f(x_1, x_2, \cdots, x_n) \in \mathbb{Z}_p[x_1, x_2, \cdots, x_n]$

$$f(x_1, \ldots, x_n) = \sum_{j=1}^{t} c_j x_1^{e_j^1} \cdots x_n^{e_j^n} = \sum_{j=1}^{t} c_j \mathcal{M}_j(x_1, \ldots, x_n), \qquad c_j \neq 0. \tag{3.2}$$

Where $\mathcal{M}_j(x_1, \ldots, x_n)$ are the monomials of f. Let $p_1, \ldots, p_n$ be distinct primes, $m_j = \mathcal{M}_j(p_1, \ldots, p_n) = p_1^{e_j^1} \cdots p_n^{e_j^n}$, and $a_i = f(p_1^i, \ldots, p_n^i) = \sum_{j=1}^{t} c_j m_j^i$. Let $\Lambda(z)$ be defined as follows:

$$\Lambda(z) = \prod_{j=1}^{t} (z - m_j) = z^t + \lambda_{t-1} z^{t-1} + \cdots + \lambda_0.$$

Theorem 3.6.1. *For a polynomial f in (3.1), and $a_i = f(p_1^i, \ldots, p_n^i)$ with distinct primes $p_1, \ldots, p_n$, the sequence $\{a_i\}_{i \geq 0}$ is linear recurrence such that, $\Lambda(z)$ is the minimal characteristic polynomial of $\{a_i\}_{i \geq 0}$ (Ben-Or and Tiwari, 1988).*

3.7 Pseudocode

Ben-Or Tiwari Algorithm

1: **Input:** Modular black box $\mathcal{B} : (\mathbb{Z}^n, p) \longrightarrow \mathbb{Z}_p$ for the target polynomial $f(x_1, \ldots, x_n)$, number of variables n, prime p.

2: **Output:** Sparse polynomial $f(x_1, \ldots, x_n)$ or **FAIL**

3: $Primes \leftarrow [2, 3, \ldots, p_n] \in \mathbb{Z}^n$
4: $eval \leftarrow [\,]$
5: $T \leftarrow 4$
6: $eval$.append($\mathcal{B}([1, \ldots, 1], p)$)
7: **while** true **do**
8: **for** $j \leftarrow 1$ **to** T **do**
9: $eval$.append($\mathcal{B}([2^j, 3^j, \ldots, p_n^j], p)$)
10: **end for**
Construct minimal characteristic polynomial via Berlekamp Massey algorithm
11: $\Lambda(z) \leftarrow$ Berlekamp Massey Euclidean Algorithm($eval$, p)
12: $terms \leftarrow \deg(\Lambda(z))$
Factor $\Lambda(z)$ to find roots using maple's Cantor-Zassenhaus algorithm
13: $roots \leftarrow$ Roots(Λ)
14: **if** $terms = |roots|$ and $terms < T$ **then**
15: **break**
16: **end if**
17: $T \leftarrow 2T$
18: **end while**
Recover monomials from roots using trial division
19: $\mathcal{M} \leftarrow$ get_monomial($roots, Primes, n, vars$)
Recover coefficients via Zippel_Vandermonde_solver
20: $A \leftarrow$ Zippel_Vandermonde_solver($eval, terms, roots, \Lambda(z), p$)
21: $f \leftarrow \sum_{i=1}^{terms} A_i \mathcal{M}_i$

Generate Monomials

Input: Roots of the minimal characteristic polynomial $roots = \{r_1, \ldots, r_{terms}\} \in \mathbb{Z}_p$, list of $Primes = [p_1, \ldots, p_n]$, list of variables $vars = [x_1, \ldots, x_n]$.
Output: Monomials $M = \{m_1, \ldots, m_{terms}\}$ where $m_i \in \mathbb{Z}[x_1, \ldots, x_n]$, or **FAIL**.
$M \leftarrow [\,]$
for $i \leftarrow 1$ **to** $terms$ **do**
    if $roots[i] = 0$ **then**
        return FAIL
    end if
    $r \leftarrow roots[i], \quad m \leftarrow 1$
    for $j \leftarrow 1$ **to** n **do**
        $e \leftarrow 0$ ▷ Exponent for x_j
        while $r \bmod p_j = 0$ **do**
            $r \leftarrow r/p_j, \quad e \leftarrow e + 1$
        end while
        $m \leftarrow m \cdot x_j^e$
    end for
    if $r \neq 1$ **then**
        return FAIL ▷ Unfactored remainder
    end if
    $M[i] \leftarrow m$
end for
return M

Zippel Transpose Vandermonde Solver

Input: Evaluation points $y = \{y_1, \ldots, y_s\} \in \mathbb{Z}_p$, number of terms s, roots $\{r_1, \ldots, r_s\} \in \mathbb{Z}_p$, characteristic polynomial $\Lambda(z) \in \mathbb{Z}_p[z]$, prime p.
Output: Coefficients $A = \{a_1, \ldots, a_s\} \in \mathbb{Z}_p$ such that $\sum_{i=1}^{s} a_i r_i^k = y_k$ for $k = 1, \ldots, s$.
$M(z) \leftarrow \Lambda(z) \bmod p$
$A \leftarrow [\,]$ ▷ Initialize coefficient vector
for $i \leftarrow 1$ **to** s **do**
    $q(z) \leftarrow \frac{M(z)}{z - r_i}$ ▷ Quotient polynomial
    $q_\Lambda^{-1} \leftarrow \frac{1}{q(r_i)} \bmod p$ ▷ Inverse evaluation
    $V_i^{-1} \leftarrow 0$ ▷ Compute i-th component of $V^{-1}y$
    for $j \leftarrow 1$ **to** s **do**
        $V_i^{-1} \leftarrow V_i^{-1} + [z^{j-1}]q(z) \cdot y_j \bmod p$ ▷ $[z^{j-1}]q(z)$ is coefficient of z^{j-1}
    end for

12: $A[i] \leftarrow V_i^{-1} \cdot q_\Lambda^{-1} \bmod p$
13: **end for**
14: **return** A

3.8 Example of Ben-Or Tiwari multivariate interpolation

Example 3. *Let $\mathcal{B}$ be the blackbox for $f(x,y,z) = x^2 + 3xy + 5yz \in \mathbb{Z}_{17}[x,y,z]$ in $lex(x > y > z)$ that we wish to recover using Ben-Or Tiwari algorithm.*
Since we have three variables, we will evaluate the blackbox $\mathcal{B}$ at $[2^i, 3^i, 5^i]$, where $0 \leq i \leq n$

$$f(x) = \begin{bmatrix} x^2 & xy & yz \end{bmatrix}^T \begin{bmatrix} 1 \\ 3 \\ 5 \end{bmatrix}$$

Finding the monomials of $f(x,y,z)$

prime_powers	Y=$\mathcal{B}(2^i, 3^i, 5^i)$
[1, 1, 1]	9
[2, 3, 5]	12
[4, 9, 8]	8
[8, 10, 6]	9
[16, 13, 13]	8
[15, 5, 14]	1

The corresponding Hankel matrix will be

$$H_3 = \begin{bmatrix} 9 & 12 & 8 \\ 12 & 8 & 9 \\ 8 & 9 & 8 \end{bmatrix} \begin{bmatrix} b_0 \\ b_1 \\ b_2 \end{bmatrix} = \begin{bmatrix} 9 \\ 8 \\ 1 \end{bmatrix}$$

Solving this system of equations give us

$$\begin{bmatrix} b_0 \\ b_1 \\ b_2 \end{bmatrix} = \begin{bmatrix} 3 \\ 13 \\ 8 \end{bmatrix}$$

The characteristic polynomial will be,

$$\Lambda(Z) = Z^3 - 3Z^2 - 13Z - 8 \ mod\ 17 = 14 + 4Z + 9Z^2 + Z^3$$

We factorize the minimal characteristic polynomial using Cantor-Zassenhaus algorithm in maple

$$\Lambda(Z) = (Z-4)(Z-6)(Z-15)$$

The roots of the minimal characteristic polynomial encode the structure of monomials, where each prime component of Prime powers correspond to each variable.

$$2 \longleftrightarrow x$$

$$3 \longleftrightarrow y$$

$$5 \longleftrightarrow z$$

Factorizing the roots of characteristic polynomials

$$4 = 2^2 \longleftrightarrow x^2, \quad 6 = 2.3 \longleftrightarrow xy, \quad 15 = 3.5 \longleftrightarrow yz$$

$$f(x,y,z) = \begin{bmatrix} x^2 & xy & yz \end{bmatrix} \begin{bmatrix} c_0 \\ c_1 \\ c_2 \end{bmatrix}$$

We use Zippel's transpose Vandermonde Solver to find the coefficients of $f(x,y,z)$

$$M(Z) = 14 + 4Z + 9Z^2 + Z^3$$

$$l_1(Z) = \frac{(Z-6)(Z-15)}{(4-6)(4-15)} \quad l_2(Z) = \frac{(Z-4)(Z-15)}{(6-4)(6-15)} \quad l_3(Z) = \frac{(Z-4)(Z-6)}{(15-4)(15-6)}$$

$$l_1(Z) = \frac{90 - 22Z + Z^2}{22} \ mod\ 17 = 1 + 6Z + 7Z^2$$

$$l_2(Z) = \frac{60 - 19Z + Z^2}{-18} \ mod\ 17 = 8 + 2Z + 16Z^2$$

$$l_3(Z) = \frac{24 - 10Z + Z^2}{99} \ mod\ 17 = 9 + 9Z + 11Z^2$$

$$\begin{bmatrix} 1 & 6 & 7 \\ 8 & 2 & 16 \\ 9 & 9 & 11 \end{bmatrix} \begin{bmatrix} 9 \\ 12 \\ 8 \end{bmatrix} = \begin{bmatrix} 1 \\ 3 \\ 5 \end{bmatrix}$$

$$\implies f(x,y,z) = \begin{bmatrix} x^2 & xy & yz \end{bmatrix} \begin{bmatrix} 1 \\ 3 \\ 5 \end{bmatrix} = x^2 + 3xy + 5yz$$

3.9 Failure probability

Theorem [8] If $p_1, \ldots, p_n$ are chosen randomly and uniformly from a subset S of the domain, which is assumed to be an integral domain, then for the sequence $\{a_i\}_{i \geq 1}$, where $a_i = f(p_1^i, \ldots, p_n^i)$, the Berlekamp–Massey algorithm encounters a singular Hankel matrix (and the corresponding zero discrepancy) the first time at $N = 2t + 1$ with probability no less than

$$1 - \frac{t(t+1)(2t+1)\deg(f)}{6 \cdot |S|},$$

where $|S|$ is the number of elements in S.

From the theorem, we have

$$\Pr[\text{success at } N = 2t+1] \;\geq\; 1 - \frac{t(t+1)(2t+1)\ \deg(f)}{6\,|S|}.$$

In our case, we are using $S = \mathbb{F}_p$ where $p = 2^{31} - 1$, therefore, the failure probability satisfies

$$\Pr[\text{failure}] \;\leq\; \frac{t(t+1)(2t+1)\ \deg(f)}{6\ \cdot 2^{31} - 1}.$$

Chapter 4

Sparse Multivariate Rational function interpolation

4.1 Kaltofen Yang algorithm

Given a black box for a multivariate rational function, the Kaltofen Yang algorithm [10] helps to recover the multivariate rational function up to a constant by recovering the numerator and the denominator. The following figure displays the components of the Kaltofen-Yang algorithm along with the flow of control.

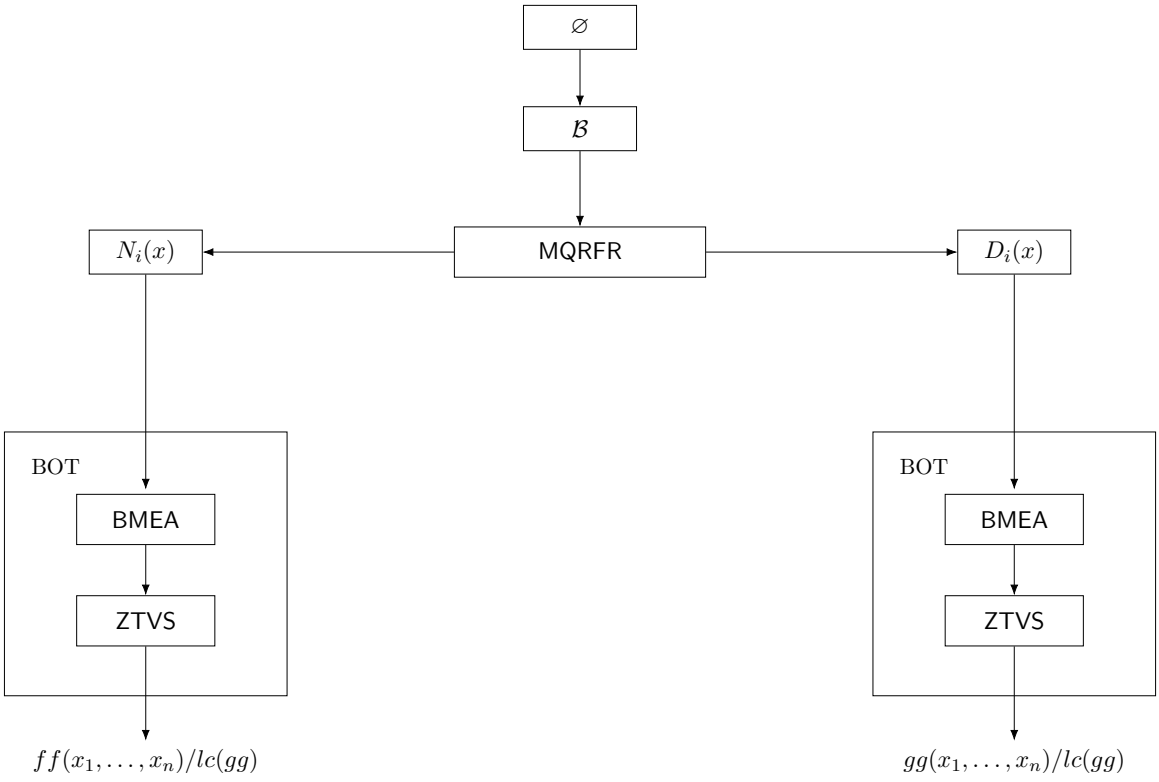


Figure 4.1: High level overview of Kaltofen Yang algorithm

Where

- $\phi : \mathbb{Z}_p(x_1, \ldots, x_n) \longrightarrow \mathbb{Z}_p(x)$
- $mathcalB$ is the black box of the multivariate rational function.
- MQRFR is the reconstruction of the maximal quotient rational function.
- N(x) and D(x) are the separated, univariate images of the numerator and denominator of the multivariate rational function, respectively.
- BOT is the Ben-Or Tiwari algorithm
- BMEA is the Berlekamp Massey Euclidean algorithm
- ZTVS is the Zippel transpose Vandermonde solver

ϕ is the homomorphism presented. To understand the flow of control presented in figure 4.1 it is easier to consider the tree-like configuration of the flow of control in reverse order, from the leaves, ie the Ben-Or Tiwari algorithm, and then explore how the MQRFR algorithm is used to bridge the black box of a multivariate rational function with the Ben-Or Tiwari algorithm.
MQRFR uses an extended Euclidean algorithm, so it can only be applied on polynomials over Euclidean domains, but the ring of multivariate polynomials is not a Euclidean domain; therefore, we cannot use MQRFR with multivariate polynomials. Therefore,to overcome this, Kaltofen and Yang define the following homomorphism and construct univariate rational images of the multivariate rational function at special points.

$$\begin{array}{ccc} \mathbb{Z}_p(x_1, \ldots, x_n) & \xrightarrow{\quad\phi\quad} & \mathbb{Z}_p(x) \\ \downarrow & & \downarrow \\ \mathbb{Z}_p(x_1, \ldots, x_n)/(x_1 - \alpha_j, \ldots, x_n - \beta_n(\alpha_j - \sigma_1) + \sigma_n) \cong \mathbb{Z}_p & & \mathbb{Z}_p(x)/(x - \alpha_j) \cong \mathbb{Z}_p \end{array}$$

where

$$\begin{array}{rccl} \phi: & \mathbb{Z}_p(x_1, \ldots, x_n) & \longrightarrow & \mathbb{Z}_p(x) \\ & x_1 & \longmapsto & x \\ & x_2 & \longmapsto & \beta_2(x - \sigma_1) + \sigma_2 \\ & \vdots & & \vdots \\ & x_n & \longmapsto & \beta_n(x - \sigma_1) + \sigma_n \end{array}$$

Evaluating the black box at points of the form $[x, \beta_2(x-\sigma_1^i)+\sigma_2^i, \ldots, \beta_n(x-\sigma_1^i)+\sigma_n^i]$ will let us reconstruct the univariate image using MQRFR.

$$T_i(x) = \frac{N_i(x)}{D_i(x)} = \frac{ff(x, \beta_2(x-\sigma_1^i)+\sigma_2^i, \ldots, \beta_n(x-\sigma_1^i)+\sigma_n^i)}{gg(x, \beta_2(x-\sigma_1^i)+\sigma_2^i, \ldots, \beta_n(x-\sigma_1^i)+\sigma_n^i)} \tag{4.1}$$

These univariate images have the following special property:

$$\implies T_i(\sigma_1^i) = \frac{N_i(\sigma_1^i)}{D_i(\sigma_1^i)} = \frac{ff(\sigma_1^i, \sigma_2^i, \ldots, \sigma_n^i)}{gg(\sigma_1^i, \sigma_2^i, \ldots, \sigma_n^i)} \tag{4.2}$$

Ben-Or Tiwari algorithm expects the following linear recurrence as input

$$\{f(\sigma_1^0, \ldots, \sigma_n^0), \ldots, f(\sigma_1^{m-1}, \ldots, \sigma_n^{m-1})\}$$

From equation 4.2 we know that the above sequence will be identical to the following sequence,

$$\{T_0(\sigma_1^0), T_1(\sigma_1^1), \ldots, T_{m-1}(\sigma_1^{m-1})\}.$$

Since $T_i(x)$ is a univariate polynomial, we can reconstruct it by separating the numerator and the denominator using MQRFR. We will get two sequences $\{N_i(\sigma_1^i)\}$ and $\{D_i(\sigma_1^i)\}$ for the numerator and denominator, respectively. We then used Ben-Or Tiwari with each sequence to recover the corresponding multivariate polynomials.

4.2 Pseudocode

Multivariate Rational Function Interpolation

1: **Input:** Modular black box $\mathcal{B} : (\mathbb{Z}^n, p) \longrightarrow \mathbb{Z}_p$ for a rational function $\frac{ff(x_1,\ldots,x_n)}{gg(x_1,\ldots,x_n)}$ over $\mathbb{Z}_p$, where $ff, gg \in \mathbb{Z}[x_1, \ldots, x_n]$, $\gcd(ff, gg) = 1$, gg is monic, prime p, number of variables n and list of variables $vars$.
2: **Output:** $ff(x_1, \ldots, x_n), gg(x_1, \ldots, x_n)$ or **fail**.
3: $Primes \leftarrow [2, 3, \ldots, p_n] \in \mathbb{Z}^n$
4: Pick random vector $\beta = [\beta_2, \ldots, \beta_n] \in \mathbb{Z}_p^{n-1}$
5: $num \leftarrow [\,]$, $den \leftarrow [\,]$
6: $num\_points\_mqrfr \leftarrow 0$
7: $numerator\_done \leftarrow$ false
8: $denominator\_done \leftarrow$ false
9: $T\_old \leftarrow 1$
10: $T \leftarrow 4$
11: $\sigma^0 \leftarrow [1, \ldots, 1] \in \mathbb{Z}_p^n$
12: $(f_0, g_0) \leftarrow \text{NDSA}(\mathcal{B}, \sigma^0, \beta, p, T)$ where, $f_0, g_0 \in Z_p[x]$

13: num.append$(f_0(\sigma_1^0) \bmod p)$
14: den.append$(g_0(\sigma_1^0) \bmod p)$
15: $num\_points\_mqrfr \leftarrow \deg(f_0) + \deg(g_0) + 2$
16: **while** true **do**
17: **for** $j \leftarrow T\_old$ to $2T - 1$ **do**
18: $\sigma^j \leftarrow [2^j, 3^j, \ldots, p_n^j] \mod p$
19: $(f_j, g_j) \leftarrow \text{NDSA}(\mathcal{B}, \sigma^j, \beta, p, num\_points\_mqrfr)$
20: **if** *not numerator_done* **then**
21: num.append$(f_j(\sigma_1^j) \bmod p)$
22: **end if**
23: **if** *not denominator_done* **then**
24: den.append$(g_j(\sigma_1^j) \bmod p)$
25: **end if**
26: **end for**
Construct minimum characteristic polynomials using Berlekamp-Massey algorithm
27: **if** *not numerator_done* **then**
28: $\Lambda_{num}(z) \leftarrow \text{Berlekamp\_Massey}(num, p, z) \in \mathbb{Z}_p[z]$
29: $s \leftarrow \deg(\Lambda_{num}(z))$
30: Let $roots_{num}$ be the distinct roots of $\Lambda_{num}(z) \in \mathbb{Z}_p[z]$.
31: **end if**
32: **if** *not denominator_done* **then**
33: $\Lambda_{den}(z) \leftarrow \text{Berlekamp\_Massey}(den, p, z) \in \mathbb{Z}_p[z]$
34: $d \leftarrow \deg(\Lambda_{den}(z))$
35: Let $roots_{den}$ be the distinct roots of $\Lambda_{den}(z) \in \mathbb{Z}_p[z]$.
36: **end if**
37: **if** $s = |roots_{num}|$ and $s < T$ **then**
38: $numerator\_done \leftarrow$ true
39: **end if**
40: **if** $d = |roots_{den}|$ and $d < T$ **then**
41: $denominator\_done \leftarrow$ true
42: **end if**
43: **if** $numerator\_done \wedge denominator\_done$ **then**
44: **break**
45: **end if**
46: $T\_old \leftarrow 2T$
47: $T \leftarrow 2T$
48: **end while**

Recover monomials from roots using trial division

$N \leftarrow$ get_monomial$(roots_{num}, Primes, n, vars)$

$D \leftarrow$ get_monomial$(roots_{den}, Primes, n, vars)$

if N = FAIL or D = FAIL **then**

    return FAIL

end if

Recover coefficients via Zippel Vandermonde solver

$A \leftarrow$ Zippel_Vandermonde_solver$(num, s, roots_{num}, \Lambda_{num}(z), p)$

$B \leftarrow$ Zippel_Vandermonde_solver$(den, d, roots_{den}, \Lambda_{den}(z), p)$

$ff \leftarrow \sum_{m=1}^{s} A_m N_m, \quad gg \leftarrow \sum_{m=1}^{d} B_m D_m$

Let μ be the leading coefficient of gg in grlex order where $x_1 > \cdots > x_n$.

$ff \leftarrow \mu^{-1} ff$ mod p, $gg \leftarrow \mu^{-1} gg$ mod p.

return ff, gg.

Numerator Denominator Separation Algorithm (NDSA)

Input: Modular black box $\mathcal{B}$ for rational function $\frac{ff(x_1,\ldots,x_n)}{gg(x_1,\ldots,x_n)}$ over $\mathbb{Z}_p$, prime p, where $ff, gg \in K[x_1,\ldots,x_n]$, $\gcd(ff, gg) = 1$, $\sigma \in \mathbb{Z}_p^n$, $\beta \in \mathbb{Z}_p^{n-1}$, $num\_points \in \mathbb{N}$.
Output: $f(x) = \frac{ff(x,\beta_2(x-\sigma_1)+\sigma_2,\ldots,\beta_n(x-\sigma_1)+\sigma_n)}{c}$, $g(x) = \frac{gg(x,\beta_2(x-\sigma_1)+\sigma_2,\ldots,\beta_n(x-\sigma_1)+\sigma_n)}{c}$, for some scalar $c \in \mathbb{Z}_p$.
$t \leftarrow num\_points$
while true **do**
    Pick random vector $\alpha = [\alpha_1,\ldots,\alpha_t] \in \mathbb{Z}_p^t$
    $m(x) \leftarrow \prod_{k=1}^{t}(x-\alpha_k) \in \mathbb{Z}_p[x]$
    $\Phi \leftarrow [\phi(\alpha_1),\ldots,\phi(\alpha_t)] \in \mathbb{Z}_p^{t\times n}$ such that: $\phi(\alpha_k) \leftarrow [\alpha_k, \beta_2(\alpha_k-\sigma_1)+\sigma_2,\ldots,\beta_n(\alpha_k-\sigma_1)+\sigma_n] \mod p \;\forall\; 1 \le k \le t$
    $Y \leftarrow [\mathcal{B}(\Phi(\alpha_1),p),\ldots,B(\Phi(\alpha_t),p)] \in \mathbb{Z}_p^t$
    $u(x) \leftarrow \text{Interpolate}(\alpha, Y, x) \mod p$
    $(f(x), g(x), deg\_q) \leftarrow \text{MQRFR}(m, u) \mod p$
    if $deg\_q > 1$ **then**
        break
    else
        $t \leftarrow 2t$
    end if
end while
return f, g

4.3 Example of Kaltofen Yang multivariate rational function interpolation

Let $\mathcal{B}$ be the blackbox for the rational function $F(x,y) = \frac{x+5y}{xy+3} \in \mathbb{Z}_{107}(x,y)$ in $lex(x > y)$ that we wish to recover using Kaltofen-Yang algorithm.

Let number of points $t = 5$. The first step is to generate the sequence $T_i(x)$ using the homomorphism

$$\begin{aligned} \phi: \quad \mathbb{Z}_{107}(x,y) &\longrightarrow \mathbb{Z}_{107}(x) \\ x &\longmapsto x \\ y &\longmapsto \beta(x-\sigma_1)+\sigma_2 \end{aligned}$$

However, since we do not know $F(x,y)$, all we can do is query the blackbox $\mathcal{B}$ at different points.

$$\text{Let } \alpha = \begin{bmatrix} \alpha_1 & \alpha_2 & \alpha_3 & \alpha_4 & \alpha_5 \end{bmatrix}^T = \begin{bmatrix} 1 & 2 & 3 & 4 & 5 \end{bmatrix}^T$$

$$\text{Let } \sigma^i = \begin{bmatrix} \sigma_1^i \\ \sigma_2^i \end{bmatrix} = \begin{bmatrix} 2^i \\ 3^i \end{bmatrix} \quad \text{and } \beta = 7$$

$$\implies T_i(x) = \mathcal{B}([x, 7(x-\sigma_1^i)+\sigma_2^i], 107) = \frac{ff(x, 7(x-\sigma_1^i)+\sigma_2^i)}{gg(x, 7(x-\sigma_1^i)+\sigma_2^i)} \bmod 107 \tag{4.3}$$

For $i = 0$ we'll have $\sigma^0 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$. Substituting σ^0 in equation 1.

$$T_0(x) = \mathcal{B}([x, 7(x-1)+1], 107) = \frac{ff(x, 7(x-1)+1)}{gg(x, 7(x-1)+1)} \bmod 107$$

Now, we want to find out $T_0(\alpha_j)$ where $1 \leq j \leq 5$ by evaluating the blackbox $\mathcal{B}$ at $[\alpha_j, 7(\alpha_j - 1) + 1)]$.

$$T_0(\alpha_j) = \mathcal{B}([\alpha_j, 7(\alpha_j-1)+1], 107) = \frac{ff(\alpha_j, 7(\alpha_j-1)+1)}{gg(\alpha_j, 7(\alpha_j-1)+1)} \bmod 107$$

Now we can use Maximal quotient rational function reconstruction algorithm to recover the

Table 4.1: Computed values of σ_1^0, σ_2^0, and $T_0(\alpha_j)$

α_j	$(\alpha_j,\ \beta(\alpha_j - \sigma_1^0) + \sigma_2^0)$	$T_0(\alpha_j) = \mathcal{B}(\alpha_j, 7(\alpha_j - 1) + 1)$
1	[1, 1]	55
2	[2, 8]	36
3	[3, 15]	15
4	[4, 22]	33
5	[5, 29]	95

univariate image of the numerator and denominator separately. The modulus polynomial is

$$\overline{m} = (x-1)(x-2)(x-3)(x-4)(x-5) = x^5 + 92x^4 + 85x^3 + 96x^2 + 60x + 94.$$

The interpolating polynomial is

$$u_0(x) = 52x^4 + 4x^3 + 66x^2 + 45x + 102,$$

Applying the maximal quotient rational function reconstruction algorithm (MQRFR) to $\overline{m}(x)$ and $u_0(x)$ modulo 107 yields the following sequence of quotients q_k, remainders r_k, and auxiliary polynomials t_k:

Table 4.2: MQRFR computations for input polynomials $\overline{m}(x)$ and $u_0(x)$

k	q_k	r_k	t_k
0	–	$x^5 + 92x^4 + 85x^3 + 96x^2 + 60x + 94$	0
1	$35x + 32$	$52x^4 + 4x^3 + 66x^2 + 45x + 102$	1
2	$52x + 21$	$x^3 + 47x^2 + 79x + 40$	$72x + 75$
3	$21x^2 + 95x + 44$	$51x + 11$	$x^2 + 45x + 31$
4	–	91	$86x^4 + 30x^3 + 59x^2 + 69x + 102$

At the step where $\deg(q_i)$ is maximal ($i = 3$), we identify

$$N_0(x) = 51x + 11, \qquad D_0(x) = x^2 + 45x + 31.$$

These are the univariate images of the rational multivariate function we are trying to recover.

$$T_0(x) = \frac{51x + 11}{x^2 + 45x + 31}$$

For this example, we will reuse the alpha values, so our modulus polynomial is the same for each iteration, only the interpolating polynomial u changes.
For $i = 1$ we'll have $\sigma^1 = \begin{bmatrix} 2 \\ 3 \end{bmatrix}$. Substituting σ^1 in equation 1.

$$T_1(x) = \mathcal{B}([x, 7(x-2)+3], 107) = \frac{ff(x, 7(x-2)+3)}{gg(x, 7(x-2)+3)} \bmod 107$$

Table 4.3: Computed values of σ_1^1, σ_2^1, and $T_1(\alpha_j)$

α_j	$[\alpha_j,\ \beta(\alpha_j - \sigma_1^1) + \sigma_2^1]$	$T_1(\alpha_j) = \mathcal{B}([\alpha_j, 7(\alpha_j - 1) + 1], 107)$
1	[1, 103]	19
2	[2, 3]	97
3	[3, 10]	47
4	[4, 17]	54
5	[5, 24]	68

The following table presents the remaining output. Where $\Phi_i = [\alpha_j,\ \beta(\alpha_j - \sigma_1^i) + \sigma_2^i]$ and $Y_i = \mathcal{B}(\Phi_i, 107)$.

Table 4.4: Values of σ_i, Φ_i, u_i, $N_i(x), D_i(x)$ and related quantities.

i	σ^i	Φ_i	Y_i	$u_i(x)$	$N_i(x)$	$D_i(x)$
1	$\begin{bmatrix}2\\3\end{bmatrix}$	$\begin{bmatrix}[1,103]\\[2,3]\\[3,10]\\[4,17]\\[5,24]\end{bmatrix}$	$\begin{bmatrix}19\\97\\47\\54\\68\end{bmatrix}$	$66x^4+102x^3+28x^2+2x+35$	$51x+38$	$x^2+29x+31$
2	$\begin{bmatrix}4\\9\end{bmatrix}$	$\begin{bmatrix}[1,95]\\[2,102]\\[3,2]\\[4,9]\\[5,16]\end{bmatrix}$	$\begin{bmatrix}66\\95\\49\\4\\99\end{bmatrix}$	$16x^4+31x^3+72x^2+105x+56$	$51x+17$	$x^2+89x+31$
3	$\begin{bmatrix}8\\27\end{bmatrix}$	$\begin{bmatrix}[1,85]\\[2,92]\\[3,99]\\[4,106]\\[5,6]\end{bmatrix}$	$\begin{bmatrix}17\\78\\68\\1\\27\end{bmatrix}$	$77x^4+17x^3+24x^2+106x+7$	$51x+71$	$x^2+57x+31$
4	$\begin{bmatrix}16\\81\end{bmatrix}$	$\begin{bmatrix}[1,83]\\[2,90]\\[3,97]\\[4,104]\\[5,4]\end{bmatrix}$	$\begin{bmatrix}77\\51\\81\\25\\29\end{bmatrix}$	$12x^4+106x^3+55x^2+64x+54$	$51x+39$	$x^2+72x+31$
5	$\begin{bmatrix}32\\29\end{bmatrix}$	$\begin{bmatrix}[1,26]\\[2,33]\\[3,40]\\[4,47]\\[5,54]\end{bmatrix}$	$\begin{bmatrix}82\\66\\6\\78\\50\end{bmatrix}$	$90x^4+21x^3+63x^2+10x+5$	$51x+90$	$x^2+18x+31$
6	$\begin{bmatrix}64\\87\end{bmatrix}$	$\begin{bmatrix}[1,74]\\[2,81]\\[3,88]\\[4,95]\\[5,102]\end{bmatrix}$	$\begin{bmatrix}34\\31\\77\\6\\69\end{bmatrix}$	$4x^4+75x^3+63x^2+79x+27$	$51x+2$	$x^2+86x+31$
7	$\begin{bmatrix}21\\47\end{bmatrix}$	$\begin{bmatrix}[1,14]\\[2,21]\\[3,28]\\[4,35]\\[5,42]\end{bmatrix}$	$\begin{bmatrix}86\\0\\41\\2\\106\end{bmatrix}$	$9x^4+36x^3+104x^2+71x+80$	$51x+5$	x^2+x+31

We evaluate the recovred numerators and denominators at different σ_1^i to generate two independent linear recurring sequences for the numerator and the denominator respectively.

$$N_i(\sigma_1^i) := [62, 33, 7, 51, 106, 10, 56, 6]$$

Table 4.5: Values of σ^i, $N_i(\sigma^i)$, and $D_i(\sigma^i)$.

i	σ^i	$N_i(\sigma_1^i)$	$D_i(\sigma_1^i)$
0	$\begin{bmatrix}1\\1\end{bmatrix}$	$51(1)+11$ =62	$(1)^2+45(1)+31=77$
1	$\begin{bmatrix}2\\3\end{bmatrix}$	$51(2)+38$ =33	$(2)^2+29(2)+31=93$
2	$\begin{bmatrix}4\\9\end{bmatrix}$	$51(4)+17=7$	$(4)^2+89(4)+31=82$
3	$\begin{bmatrix}8\\27\end{bmatrix}$	$51(8)+71=51$	$(8)^2+57(8)+31=16$
4	$\begin{bmatrix}16\\81\end{bmatrix}$	$51(16)+39=106$	$(16)^2+72(16)+31=48$
5	$\begin{bmatrix}32\\29\end{bmatrix}$	$51(32)+90=10$	$(32)^2+18(32)+31=26$
6	$\begin{bmatrix}64\\87\end{bmatrix}$	$51(64)+2=10$	$(64)^2+86(64)+31=1$
7	$\begin{bmatrix}21\\47\end{bmatrix}$	$51(21)+5=6$	$(21)^2+(21)+31=65$

$$D_i(\sigma_1^i) := [77, 93, 82, 16, 48, 26, 1, 65]$$

We use the Berlekamp Massey Euclidean algorithm to find the minimal characteristic polynomial for the sequence corresponding to numerator and the denominator.

Numerator

We compute the Syndrome polynomial using the terms of the sequence

$$S_N(Z) = 62Z^7 + 33Z^6 + 7Z^5 + 51Z^4 + 106Z^3 + 10Z^2 + 56Z + 6$$

i	q_i	r_i	t_i
0	–	Z^8	0
1	$19Z+71$	$62Z^7+33Z^6+7Z^5+51Z^4+106Z^3+10Z^2+56Z+6$	1
2	$3Z+47$	$92Z^6+32Z^5+36Z^4+95Z^3+45Z^2+83Z+2$	$88Z+36$
3	–	$Z+19$	$57Z^2+36Z+21$

$$\Lambda_N(Z) = 57Z^2 + 36Z + 21$$

Making $\Lambda_N(Z)$ monic by normalizing it gives us

$$\Lambda_N(Z) = Z^2 + 102Z + 6$$

We now find the monomials in the numerator by factorizing the minimal characteristic polynomial for the numerator using maple's Cantor-Zassenhaus algorithm

$$\Lambda_N(Z) = (Z-2)(Z-3)$$

We recover the monomials for the numerator by factorizing the roots of characteristic polynomial for the numerator

$$2 \longleftrightarrow x$$

$$3 \longleftrightarrow y$$

$$ff(x,y) = \begin{bmatrix} x & y \end{bmatrix} \begin{bmatrix} c_0 \\ c_1 \end{bmatrix}$$

We use Zippel's transpose Vandermonde Solver to find the coefficients of the numerator ff

$$M(Z) = Z^2 + 102Z + 6$$

$$l_1(Z) = \frac{Z-3}{(2-3)} \quad l_2(Z) = \frac{Z-2}{(3-2)}$$

$$l_1(Z) = 3 - Z \bmod 107 = 3 + 106Z$$

$$l_2(Z) = -2 + Z \bmod 107 = 105 + Z$$

$$\begin{bmatrix} 3 & 106 \\ 105 & 1 \end{bmatrix} \begin{bmatrix} 62 \\ 33 \end{bmatrix} = \begin{bmatrix} 46 \\ 16 \end{bmatrix}$$

$$ff(x,y) = \begin{bmatrix} x & y \end{bmatrix} \begin{bmatrix} 46 \\ 16 \end{bmatrix} = 46x + 16y$$

Denominator

$$S_D(Z) = 77Z^7 + 93Z^6 + 82Z^5 + 16Z^4 + 48Z^3 + 26Z^2 + Z + 65$$

i	q_i	r_i	t_i
0	–	Z^8	0
1	$82Z + 83$	$77Z^7 + 93Z^6 + 82Z^5 + 16Z^4 + 48Z^3 + 26Z^2 + Z + 65$	1
2	$92Z + 98$	$2Z^6 + 14Z^5 + 86Z^4 + 90Z^3 + 7Z^2 + 44Z + 62$	$25Z + 24$
3	–	$43Z + 88$	$54Z^2 + 50Z + 3$

$$\Lambda_D(Z) = 54Z^2 + 50Z + 3$$

Making $\Lambda_D(Z)$ monic by normalizing it gives us

$$\Lambda_D(Z) = Z^2 + 100Z + 6$$

We now find the monomials in the denominator by factorizing the minimal characteristic polynomials for the denominator using Maple's Cantor-Zassenhaus algorithm.

$$\Lambda_D(Z) = (Z-1)(Z-6)$$

We recover the monomials for the denominator by factorizing the roots of characteristic polynomial for the denominator

$$2 \longleftrightarrow x$$

$$3 \longleftrightarrow y$$

$$1 = 2^0 3^0 \longleftrightarrow 1, \qquad 6 = 2.3 \longleftrightarrow xy$$

$$gg(x,y) = \begin{bmatrix} 1 & xy \end{bmatrix} \begin{bmatrix} c_0 \\ c_1 \end{bmatrix}$$

We use Zippel's transpose Vandermonde Solver to find the coefficients of the denominator gg

$$M(Z) = Z^2 + 100Z + 6$$

$$l_1(Z) = \frac{Z-6}{(1-6)} \quad l_2(Z) = \frac{Z-1}{(6-1)}$$

$$l_1(Z) = \frac{-6+Z}{-5} \bmod 107 = 44 + 64Z$$

$$l_2(Z) = \frac{-1+Z}{5} \bmod 107 = 64 + 43Z$$

$$\begin{bmatrix} 44 & 64 \\ 64 & 43 \end{bmatrix} \begin{bmatrix} 77 \\ 93 \end{bmatrix} = \begin{bmatrix} 31 \\ 46 \end{bmatrix}$$

$$gg(x,y) = \begin{bmatrix} 1 & xy \end{bmatrix} \begin{bmatrix} 31 \\ 46 \end{bmatrix} = 31 + 46xy$$

Normalizing to make the denominator monic gives us

$$gg(x,y) = \frac{31 + 46xy}{46} \bmod 107 = xy + 3$$

Divding the numerator by the leading coefficient as well gives us,

$$ff(x,y) = \frac{46x + 16y}{46} \bmod 107 = x + 5y$$

The multivariate rational function will be,

$$\frac{ff(x,y)}{gg(x,y)} = \frac{x+y}{xy+3}$$

Chapter 5

Solving Parametric Linear Systems using Sparse Multivariate Rational Function Interpolation

Bibliography

[1] Richard E. Blahut. *Theory and Practice of Error Control Codes.* Addison-Wesley, Reading, MA, 1983.

[2] Daniel Boley. Vandermonde factorization of a hankel matrix? *Linear Algebra and its Applications*, 263:11–23, 1997.

[3] D.S. Dummit and R.M. Foote. *Abstract Algebra.* Wiley, 2003.

[4] Martin Fürer. Faster integer multiplication. In *STOC 2007*, pages 57–66, 2007.

[5] Keith O. Geddes, Stephen R. Czapor, and George Labahn. *Algorithms for Computer Algebra.* Springer, New York, 1992.

[6] Shuai Geng, Jie Zhang, and Yuanming Zhang. On hankel operators on mixed norm spaces and a related operator equation. *arXiv preprint arXiv:2206.04126*, 2022.

[7] David Harvey and Joris van der Hoeven. Integer multiplication in time $o(n \log n)$. *Annals of Mathematics*, 193(2):563–617, 2021.

[8] Erich Kaltofen and Wen-shin Lee. Early termination in sparse interpolation algorithms. *Journal of Symbolic Computation*, 36(3–4):365–400, 2003.

[9] Erich L. Kaltofen and Barry M. Trager. Computing with polynomials given by black boxes for their evaluations: Greatest common divisors, factorization, separation of numerators and denominators. *Journal of Symbolic Computation*, 9(3):301–320, 1990.

[10] Erich L. Kaltofen and Zhengfeng Yang. On exact and approximate interpolation of sparse rational functions. In *Proceedings of the 2007 International Symposium on Symbolic and Algebraic Computation (ISSAC '07)*, pages 203–210, Waterloo, Ontario, Canada, 2007. ACM.

[11] A. Karatsuba and Y. Ofman. Multiplication of many-digital numbers by automatic computers. *Doklady Akademii Nauk SSSR*, 145:293–294, 1962.

[12] Donald E. Knuth. *The Art of Computer Programming, Vol. 2: Seminumerical Algorithms.* Addison–Wesley, 3rd edition, 1997.

[13] Martin Kronenburg. Toom-cook multiplication: Some theoretical and practical aspects. *arXiv preprint arXiv:1602.02740*, 2016.

[14] Michael Monagan. Maximal quotient rational reconstruction: An almost optimal algorithm for rational reconstruction. In *Proceedings of the 2004 International Symposium on Symbolic and Algebraic Computation (ISSAC 2004)*, pages 243–249, New York, NY, USA, 2004. ACM.

[15] Jean-Michel Muller. *Elementary Functions: Algorithms and Implementation.* Birkhäuser / Springer, Boston, MA, 3 edition, 2016.

[16] Nicholson. 7.5: More on linear recurrences. `https://math.libretexts.org/Bookshelves/Linear_Algebra/Linear_Algebra_with_Applications_(Nicholson)/07:Linear_Transformations/7.05:More_on_Linear_Recurrences`, 2023. Part of the book "Linear Algebra with Applications (Nicholson)", Mathematics LibreTexts, Page ID: 58873.

[17] Arnold Schönhage and Volker Strassen. Schnelle multiplikation grosser zahlen. *Computing*, 7:281–292, 1971.

Appendix A

Code

Appendices should be used for supplemental information that does not form part of the main research. Remember that figures and tables in appendices should not be listed in the List of Figures or List of Tables.